



M Ű E G Y E T E M 1 7 8 2

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Molnár Levente

ANDROID ALAPÚ SZOFTVERFEJLESZTÉS

Háztartásvezetést támogató alkalmazás fejlesztése

KONZULENS

Pomázi Krisztián Dániel

BUDAPEST, 2025

Tartalomjegyzék

Összefoglaló	6
Abstract.....	7
1 Bevezetés	8
1.1 Motiváció	8
1.1.1 Előzetes tapasztalatok	8
1.2 A projekt célja.....	8
1.3 Követelmények elemzése	9
2 Technológiák	11
2.1 Felhasználói felület	11
2.1.1 Stílusok és témák	11
2.1.2 Navigáció	11
2.1.3 Képek kezelése	11
2.1.4 Widget.....	12
2.1.5 Animációk.....	12
2.1.6 Diagramok	12
2.2 Felhasználó- és adatkezelés	12
2.2.1 Firebase.....	12
2.2.2 Supabase	13
2.2.3 Autentikáció.....	14
2.2.4 Lokális cache	14
2.2.5 Alkalmazás beállítások	14
2.3 Kamera kezelés	15
2.4 Egyéb függőségek.....	15
2.4.1 Hálózati kommunikáció	15
2.4.2 Generatív AI	15
2.4.3 Exportálás táblázatként.....	16
2.4.4 Háttérfolyamatok	16
2.4.5 Függőséginjektálás.....	16
2.4.6 Tesztelés.....	16
2.4.7 Folyamatos integráció.....	17
3 Architektúra	18

3.1 Clean Architecture	18
3.2 Model-View-ViewModel.....	19
3.3 Az alkalmazás felépítése.....	20
3.3.1 Adatforrások	21
3.3.2 Adatkezelés	27
3.3.3 Üzleti logika.....	28
3.3.4 Felhasználói felület	29
4 Képernyők, funkciók	30
4.1 Felhasználókezelés.....	30
4.1.1 Bejelentkezés, regisztráció.....	30
4.1.2 Profilképernyő	31
4.1.3 Beállítások	32
4.2 Háztartások	33
4.3 Bevásárlólista.....	34
4.4 Készletek.....	37
4.4.1 Diagramok	39
4.4.2 Készletelőzmények	40
4.4.3 Emlékeztetők	41
4.4.4 Receptgenerálás	43
4.5 Termékek	44
4.5.1 Termék beküldés.....	45
4.5.2 Termék hitelesítés	46
4.6 Szkennelő képernyők.....	47
4.7 Alkalmazáson belüli értesítések.....	48
4.7.1 Háztartási meghívók	49
4.7.2 Termék beküldések	50
4.8 Kiegészítő funkciók	50
4.8.1 App shortcut.....	50
4.8.2 Bevásárlólista widget.....	50
4.8.3 Push értesítések.....	51
4.8.4 Vendégfelhasználók kezelése	51
4.9 Offline működés	53
5 Tesztelési stratégiák.....	55
5.1 Platformfüggetlen tesztek	55

5.1.1 Egységtesztek.....	55
5.1.2 Integrációs tesztek.....	55
5.2 Felhasználói felület tesztek	56
5.3 Folyamatos integráció.....	57
6 Összegzés.....	59
6.1 A projekt jelenlegi állapota	59
6.2 Projekt értékelése	59
6.3 Továbbfejlesztési lehetőségek	60
6.3.1 Papírra írt bevásárlólista digitalizálása	60
6.3.2 Készletelemek felvétele digitális nyugta alapján.....	61
6.3.3 Akciós termékek figyelése.....	61
6.3.4 Általános chat funkció mesterséges intelligenciával	61
6.3.5 Offline működés kiterjesztése.....	61
Irodalomjegyzék.....	63

HALLGATÓI NYILATKOZAT

Alulírott **Molnár Levente**, szigorló hallgató kijelentem, hogy ezt a diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 12. 12.

.....
Molnár Levente

Összefoglaló

A mindennapok során gyakran jelent kihívást a háztartásokban található termékek átlátható módon történő nyilvántartása, a készletek rendszerezése, valamint a lejáratí idők nyomon követése. A készletek rendszerezésének hiánya könnyen vezethet felesleges vásárlásokhoz, termékhiányhoz vagy akár lejárt termékek felhalmozódásához is. Ezen problémák kezelésére kínál megoldást az általam készített, háztartásvezetést támogató mobilalkalmazás, melynek célja a készletek egyszerű, ugyanakkor hatékony és strukturált módon történő nyilvántartása és rendszerezése.

Az alkalmazás lehetővé teszi közösen szerkeszthető háztartások létrehozását, amelyekben több felhasználó együtt kezelheti a készletelemeket és a bevásárlólistát. A rendszer alapját egy közösségi termékadatbázis képezi, amelyet a felhasználók folyamatosan bővíthetnek. Új elemek rögzítésekor a termékek azonosítása gyorsan elvégezhető az eszköz kamerájának segítségével, mivel az alkalmazás támogatja a vonalkódok beolvasását, valamint ahol szükséges, a lejáratí dátumok szkennelését is. A megadott lejáratí idők alapján a felhasználók értesítéseket állíthatnak be, melyek segítségével időben tájékozódhatnak a közelgő lejáratokról.

A fejlesztés megkezdése előtt áttekintettem a jelenleg elérhető alkalmazásokat, azok fő funkcióit és korlátait, valamint informális visszajelzéseket gyűjtöttem ismerősöktől arról, hogy milyen funkciókat tartanának hasznosnak egy ilyen alkalmazásban. Ezek a tapasztalatok is hozzájárultak az alkalmazás követelményeinek meghatározásához és a fejlesztés irányvonalához.

A projekt eredményeként egy olyan komplex, többfelhasználós mobilalkalmazás jött létre, amely egyesíti a bevásárlólista-kezelést, a készletnyilvántartást és a termékadatbázis kezelését. A rendszer számos kényelmi funkciót is tartalmaz, mint például a hangalapú termékkeresés, a kezdőképernyőre helyezhető bevásárlólista-widget, valamint a készletek állapotát szemléltető diagramok és statisztikák. Ezen funkciók együttesen egy könnyen használható, rugalmas, illetve a későbbiekben tovább bővíthető megoldást nyújtanak a háztartásokban lévő készletek rendszerezett nyilvántartására.

Abstract

Managing household items in a transparent and organized manner, keeping track of inventory levels, and monitoring expiration dates often present challenges in everyday life. The lack of proper inventory organization can easily lead to unnecessary purchases, product shortages, or the accumulation of expired goods. To address these issues, I developed a mobile application designed to support household management by enabling simple yet efficient and structured tracking and organization of household inventories.

The application allows users to create collaboratively editable households, where multiple users can jointly manage inventory items and shopping lists. Its core relies on a community-based product database that can be continuously expanded by the users. When adding new items, products can be identified quickly through the device's camera, as the application supports barcode scanning and, where applicable, expiration-date recognition. Based on the provided expiration dates, users can configure notifications that help them stay informed about items nearing expiration.

Before starting development, I reviewed currently available applications, their main features and limitations, and collected informal feedback from acquaintances regarding what functionalities they would find useful in such an application. These insights contributed to defining the system requirements and the direction of the development process.

As a result of the project, a complex, multi-user mobile application was created, integrating shopping list management, inventory tracking, and product database handling. The system includes several convenience features, such as voice-based product search, a home-screen widget for the shopping list, and diagrams and statistics that visualize inventory status. Together, these functionalities provide an easy-to-use, flexible, and future-expandable solution for the structured management of household supplies.

1 Bevezetés

Az általam választott téma keretein belül egy háztartásvezetést támogató Android alkalmazás elkészítésén dolgoztam, ebben a fejezetben szeretném ismertetni a kezdeti motivációt, a projekt célját, valamint a feladat során definiált követelményeket.

1.1 Motiváció

A mindennapokban sokan küzdenek azzal a kihívással, hogy átláthatóan nyomon kövessék, milyen termékek állnak rendelkezésre a háztartásban, és melyek azok, amelyek már elfogytak. A készletek rendszerezésének hiánya könnyen vezethet felesleges vásárlásokhoz, termékek hiányához vagy akár lejárt termékek felhalmozódásához is. Ezen problémák összességére próbál megoldást kínálni az általam fejlesztett alkalmazás, amely segít a háztartásban lévő készletek hatékony kezelésében és rendszerezésében.

1.1.1 Előzetes tapasztalatok

A konkrét feladat meghatározása előtt megkérdeztem a környezetemben élőket, hogy használnak-e ilyen típusú alkalmazást, ha igen, mit szeretnek benne, illetve mit hiányolnak belőle. Amennyiben nem, akkor használnának-e, illetve milyen funkciók lennének azok, amiket szívesen látnának egy ilyen alkalmazásban. Ezen véleményeket, ötleteket és ajánlásokat is igyekeztünk figyelembe venni a konzulensemmel a feladat követelményeinek kidolgozása során.

A felhasználói igények felmérése mellett igyekeztem utánanézni, hogy milyen meglévő megoldások érhetők el a különböző alkalmazásboltokban. Az élelmiszerláncok saját alkalmazásai lehetőséget adnak bevásárlólista készítésére, viszont a megvásárolt termékek nyilvántartására nem. Egy hasonló alkalmazást találtam, amely kifejezetten a háztartásban tárolt készletek rendszerezésére készült, viszont ez időközben megszűnt és már nem elérhető a Google Play Áruházban sem. Természetesen biztosan léteznek további alternatívák, viszont én első körben ezeket találtam meg.

1.2 A projekt célja

Az általam elkészített készletnyilvántartó alkalmazás célja főként az, hogy egyesítse a jelenleg elérhető alkalmazások előnyeit, mint a bevásárlólista készítés, illetve

a megvásárolt termékek nyilvántartása. Ezen túl viszont további olyan funkciókkal is rendelkezzen, melyek egyszerűbbé, kényelmesebbé teszik a rendszerezést.

Fontosnak tartottam továbbá, hogy a termékek ne csak egyszerű, általános elemekként legyenek eltárolva, hanem konkrét, valós termékekhez legyenek kötve. A pontos meghatározásra többek között azért is van szükség, hogy a felhasználóknak lehetősége legyen a vonalkódok és lejárat dátumok beszkenyelésére a mobil kamerájának segítségével.

1.3 Követelmények elemzése

Az alkalmazás alapját a bevásárlólista, a készletek, és a termékek kezelése adja. Az ezekhez tartozó adatok tárolása és rendelkezésre állása kulcsfontosságú a felhasználói élmény szempontjából.

Az alkalmazás több különböző autentikációs lehetőséget biztosít a felhasználóknak. Ennek célja, hogy mindenki azt a belépési formát választhassa, amely számára a legkényelmesebb. Az anonim bejelentkezés lehetőséget biztosít az alkalmazás gyors kipróbálására, az e-mail és jelszó alapú regisztráció pedig biztonságos és megszokott megoldást nyújt. Emellett a Google Auth integrálása kényelmesebb alternatívát kínál, amely lehetővé teszi a gyors bejelentkezést anélkül, hogy újabb jelszót kellene megjegyezni.

Az alkalmazásban a felhasználók több háztartást is létrehozhatnak, melyeket megoszthatnak más felhasználókkal. Ezáltal lehetőség nyílik arra, hogy az egy háztartásban élők közösen kezeljék a készleteiket és bevásárlólistáikat, így növelve a szervezettséget és csökkentve a párhuzamos vagy felesleges vásárlások esélyét.

Az értesítések fontos szerepet töltenek be a felhasználók tájékoztatásában, különösen a termékek lejárat idejével kapcsolatban. Segítenek abban, hogy a felhasználók időben információt kapjanak a közelgő lejáratokról, ezzel csökkentve az élelmiszerpazarlást és a kapcsolódó anyagi veszteségeket.

A világos és sötét téma támogatása manapság alapvető elvárás a felhasználók részéről. Ez nemcsak a kényelmet szolgálja, hanem hozzájárul a jobb olvashatósághoz és az eszköz energiafogyasztásának optimalizálásához is. A többnyelvűség támogatása szintén kiemelten fontos, mivel hozzájárul ahhoz, hogy az alkalmazás szélesebb felhasználói rétegeket is elérjen.

A kényelmi funkciók célja, hogy növeljék az alkalmazás használhatóságát és csökkentsék az elvégzendő műveletekhez szükséges időt. A hangalapú termékkeresés alternatívát kínál gyors adatbevitelre, az alkalmazás gyorselérési funkciói (app shortcut) és a bevásárlólista widget pedig segítik, hogy a felhasználók egy mozdulattal hozzáférhessenek a legfontosabb funkciókhoz. Mivel az alkalmazás használata általában rövid, célirányos interakciókat jelent, kiemelten fontos, hogy minden funkció gyorsan és könnyedén elérhető legyen.

A mesterséges intelligencia alapú receptgenerálás funkció lehetőséget biztosít arra, hogy a felhasználók az otthon található alapanyagok alapján új ételötleteket kapjanak. Ez a funkció egyrészt inspirációként szolgálhat, másrészt segít a készletek hatékonyabb felhasználásában is, így csökkentve a pazarlást.

A termékek árának rögzítése lehetőséget ad arra, hogy a felhasználók már a bevásárlólista összeállításakor közelítő költségbeclést kapjanak, ami segíti a tudatosabb vásárlást. Az alkalmazás ezen felül különböző statisztikai kimutatásokat és diagramokat biztosít, amelyek vizualizálják a készletek alakulását.

A fejlesztés során a megbízhatóság egyik alapját a megfelelő tesztelés adja. Az automatizált és manuális tesztek egyaránt fontosak annak biztosításához, hogy az alkalmazás minden funkciója helyesen működjön. Ez elengedhetetlen a pozitív felhasználói megítéléshez, valamint az alkalmazás hosszú távú sikeréhez.

2 Technológiák

Ebben a fejezetben szeretném ismertetni az alkalmazás fejlesztése során felhasznált technológiákat, külső könyvtárakat, függőségeket.

2.1 Felhasználói felület

Az alkalmazás felhasználói felületének fejlesztéséhez a Jetpack Compose technológiát használtam, amely az Android modern, deklaratív UI-fejlesztési keretrendszere. A Compose lehetővé teszi, hogy a felületek egyszerűen, kód alapú komponensekkel legyenek leírva. A komponensek újrafelhasználhatósága és testreszabhatósága nagyban megkönnyíti a fejlesztést.[1]

2.1.1 Stílusok és témák

A felhasználói felület egységes megjelenéséről a Material 3 dizájnirányelvek gondoskodnak, amelyek a modern vizuális trendeket követve biztosítják az intuitív és felhasználóbarát élményt. A Material 3 segítségével az alkalmazás színeit és stílusait egy központi rendszer alapján terveztem meg.[2]

Az alkalmazás által használt betűtípust a Google Fonts biztosítja, amely szintén összhangban van a Material 3 irányelvekkel.[3]

2.1.2 Navigáció

Az alkalmazás különböző képernyői közötti navigációt a Jetpack Compose Navigation komponenssel valósítottam meg, amely kifejezetten a Jetpack Compose-hoz készült navigációs megoldás. Ez a komponens deklaratív módot kínál az alkalmazás különböző útvonalainak kezelésére, megkönnyítve a képernyők közötti adatkommunikációt és a navigáció logikájának felépítését, illetve támogatja a navigációs animációkat is.[4]

2.1.3 Képek kezelése

A képek felhasználói felületen való megjelenítését a Coil könyvtár segítségével valósítottam meg, amely támogatja a képek hálózaton keresztül történő, aszinkron betöltését is.[5]

2.1.4 Widget

A kezdőképernyőre elhelyezhető widget elemet a Jetpack Glance könyvtár segítségével készítettem el. A Jetpack Glance egy deklaratív UI-keretrendszer, amely lehetővé teszi, hogy Jetpack Compose-hoz hasonló szintaxissal hozzunk létre egyszerű, de testreszabható widgeteket Androidon. A widgetek célja, hogy gyors hozzáférést biztosítsanak fontos funkciókhoz anélkül, hogy meg kellene nyitni az alkalmazást. Előnye továbbá, hogy a widget megjelenítése dinamikusan frissül a háttérben zajló adatváltozások alapján, és figyelembe veszi a világos és sötét témához kapcsolódó megjelenítési beállításokat is.[6]

2.1.5 Animációk

Az alkalmazásban a Jetpack Compose által biztosított animációs lehetőségek mellett a Lottie Compose külső könyvtárat is felhasználtam látványosabb, testreszabható animációk megjelenítéséhez. A Lottie lehetővé teszi JSON formátumban tárolt animációk egyszerű integrálását, ezáltal vizuálisan gazdagabb felhasználói élményt nyújtva minimális erőforrásigény mellett.[7]

2.1.6 Diagramok

A háztartásban található készletelemek alapján előállított diagramok vizuális megjelenítéséhez a Charts könyvtárat használtam fel, amely több különböző diagram típust is támogat, beleértve az általam használt kör-, illetve oszlopdiagramot is.[8]

2.2 Felhasználó- és adatkezelés

A felhasználók, valamint az alkalmazásban létrehozott adatok tárolásáról két felhőalapú szolgáltatás gondoskodik.

2.2.1 Firebase

A felhasználókhoz és háztartásaikhoz tartozó adatokat a Firebase[9] tárolja. A Firebase egy modern, felhőalapú platform, amelyet a Google fejlesztett ki, és amely számos eszközt és szolgáltatást kínál a mobil- és webalkalmazások fejlesztéséhez. Központi eleme egy valós idejű adatbázis, a Firebase Firestore[10], amely lehetővé teszi az adatok gyors és biztonságos tárolását, valamint szinkronizálását a felhasználók között.

A Firestore egy NoSQL[11] alapú, dokumentumorientált adatbázis, amelyben az adatok kollekciókba és dokumentumokba rendezve vannak eltárolva. Ez a struktúra nagymértékű rugalmasságot biztosít az adattárolás során, mivel nem igényel előre definiált séma szerinti táblákat, így az adatszerkezet könnyen módosítható és bővíthető a későbbiekben is.

A Firestore egyik legnagyobb előnye, hogy valós idejű szinkronizációt tesz lehetővé a klienseszközök között, így például, ha egy háztartás tagja módosítja a készletek adatait vagy a bevásárlólistát, akkor a módosítás azonnal megjelenik a többi felhasználónál is. Emellett a Firestore offline működést is támogat, vagyis az adatok ideiglenesen a készüléken kerülnek tárolásra, majd az internetkapcsolat helyreállásakor automatikusan szinkronizálódnak a felhővel, biztosítva a folyamatos működést hálózati hiba esetén is.

A Firebase további előnye, hogy egyszerűen integrálható a mobilalkalmazásokkal, valamint felhasználóbarát webes felületet és részletes dokumentációt biztosít a fejlesztők számára. Mivel tanulmányaim során több alkalommal is használtam már a Firebase-t különböző projektek megvalósítása során, az integráció és a beállítások elvégzése nem jelentett nehézséget.

2.2.2 Supabase

Az alkalmazásban található, minden felhasználó által elérhető adatok, illetve az alkalmazásban megjelenő képek a Supabase[12] segítségével vannak eltárolva. A Supabase egy nyílt forráskódú backend platform, amely egyszerű és hatékony megoldást kínál adatbázisok kezelésére, hitelesítésre, valamint fájlok tárolására.

Az alkalmazáshoz kapcsolódóan a Supabase-en belül a Supabase Database[13] tárolja a közösen használt termékekhez tartozó információkat. A Supabase Database egy PostgreSQL[14] alapú relációs adatbázis, amely valós idejű adatkezelést és SQL-alapú lekérdezéseket tesz lehetővé.

A Supabase Storage[15] szolgáltatás biztonságos és skálázható fájltárolási rendszert biztosít. A rendszerben külön tárolók (bucket-ek) hozhatók létre, melyeken belül komplex könyvtárszerkezet építhető mappákból és a bennük található fájlokból. Minden fájlhoz tartozik egyedi URL, amely lehetővé teszi, hogy a felhasználók által feltöltött képek, például termékfotók, könnyen elérhetők legyenek. Emellett hozzáférési

szabályok is definiálhatók, melyek korlátozzák, hogy milyen jogosultságú felhasználók érhetik el az egyes tárolóban található fájlokat.

A Supabase előnyei továbbá a Firebase-zel szemben, hogy ingyenes lehetőséget biztosít szerveroldali metódusok és adatbázis triggerek definiálására, melyekkel automatizált műveletek hajthatók végre közvetlenül az adatbázisban. Emellett a kliensoldali lekérdezések is rugalmasabbak, mivel többféle szűrési és rendezési lehetőséget támogatnak, ezáltal pontosabban és hatékonyabban kezelhetők az adatok.

2.2.3 Autentikáció

Az alkalmazásban a felhasználók autentikációját a Firebase Authentication[16] rendszer végzi, amely kézenfekvő választás, hiszen az adattárolás szintén a Firebase-ben került megvalósításra. Az autentikáció mellett támogatja a felhasználók jogosultságainak kezelését is, melynek köszönhetően minden felhasználó csak a jogainak megfelelő adatokhoz fér hozzá.

A Google Auth integrálásához szükségem volt még a Credential Manager használatára is, amely lehetővé teszi különböző hitelesítési módok, többek között a harmadik féltől származó szolgáltatások, mint például a Google kezelését egységes interfészen keresztül.[17]

2.2.4 Lokális cache

Annak érdekében, hogy az alkalmazásban tárolt adatok hálózati kapcsolat nélkül is megjelenjenek, a Firebase automatikusan gyorsítótárazza a lekérdezett adatokat. Ellenben a Supabase-ről lekérdezett információkat szükséges manuálisan eltárolni az eszközön. Ennek megvalósításához a Room[18] könyvtárat használtam fel. A Room egy, az Android által támogatott ORM (Object Relational Mapping)[19] megoldás, amely lehetővé teszi relációs adatok egyszerű és típusbiztos kezelését SQLite[20] alapokon. Segítségével a lekérdezett adatok egy lokális adatbázisban kerülnek eltárolásra.

2.2.5 Alkalmazás beállítások

Az alkalmazásban elérhető beállítások állapotát, mint például az alkalmazás nyelve vagy témája, a Jetpack DataStore kezeli. Ez az eszköz lehetővé teszi az adatok aszinkron, típusbiztos és perzisztens tárolását, így ezek az alkalmazás minden indításakor automatikusan betöltődnek.[21]

2.3 Kamera kezelés

A vonalkódok és a lejárat dátumok beszkeneléséhez szükségem volt a Camera 2 API használatára, amely az Android rendszerek modern kameravezérlési keretrendszere. A Camera 2 API lehetőséget biztosít arra, hogy az alkalmazás közvetlenül vezérelje az eszköz kameráját, illetve támogatja a valós idejű képfeldolgozást, amely elengedhetetlen a vonalkódok és dátumok azonnali feldolgozásához.[22]

A különböző beszkenelt adatok feldolgozását az ML Kit segítségével implementáltam. Az ML Kit a Google által biztosított gépi tanulási fejlesztői csomag, melynek segítségével közvetlenül az Androidos eszközön elvégezhetők különböző képfeldolgozási műveletek. Mivel a feldolgozás valós időben történik, nincs szükség a rögzített képek elmentésére sem, hiszen az ML Kit azonnal képes kinyerni a releváns információkat az aktuálisan beolvasott képből.[23]

2.4 Egyéb függőségek

Ezen alfejezetben szeretnék még megemlíteni további olyan külső könyvtárakat, melyeket felhasználtam a fejlesztés során.

2.4.1 Hálózati kommunikáció

Egy vonalkód feldolgozását követően az alkalmazás először a saját adatbázisában próbálja megkeresni a hozzá tartozó terméket. Ha ez nem jár sikerrel, egy külső adatbázisból is megpróbálja lekérdezni. Ehhez a kiegészítő funkcióhoz szükségem volt a Retrofit 2 használatára. A Retrofit 2 egy hatékony és rugalmas HTTP kliens könyvtár az Android fejlesztéshez, amely leegyszerűsíti az API-k használatát. Segítségével gyors és megbízható hálózati kommunikáció valósítható meg, illetve a lekérdezések során gondoskodik az adatok automatikus átalakításáról is.[24]

2.4.2 Generatív AI

A mesterséges intelligencia alapú receptgenerálás funkciót annak elkészítésekor még a Generative AI[25] függőség használatával valósítottam meg. A későbbiekben viszont ennek támogatottsága megszűnt, ezért áttértem az újonnan megjelent Firebase AI Logic[26] megoldásra. Ez utóbbinak nagy előnye, hogy a Firebase-en keresztül érhető el a Gemini API is. Mivel a projektben egyébként is használtam a Firebase szolgáltatásait, ezáltal az új megoldás bevezetése nem okozott gondot, illetve növelte is az átláthatóságot,

hiszen így a mesterséges intelligenciához tartozó adatforgalmat is nyomon tudtam követni a Firebase-en keresztül.

Mind a Generative AI, mint a Gemini API nagy előnye, hogy az AI által adott válasz formátumát egy sémában lehet definiálni, így könnyen fel tudtam dolgozni a visszakapott recepteket. Emellett mindkét megoldás támogat több modellt is, kezdetben a *Gemini 2.0 Flash-Lite*, a későbbiekben pedig a *Gemini 2.5 Flash-Lite* modellt használtam.[27]

2.4.3 Exportálás táblázatként

A készletelőzményekben található elemeket lehetőség van táblázatként is exportálni, ezáltal a felhasználók strukturáltabb formában is megtekinthetik azokat. A táblázat előállításához a nyílt forráskódú Apache POI könyvtár OOXML modulját használtam fel, amely támogatja a modern, XML-alapú Office formátumokat, beleértve az általam használt *.xlsx* kiterjesztést is.[28]

2.4.4 Háttérfolyamatok

A bevásárlólista időnkénti frissítését, illetve az értesítések kiküldését WorkManager segítségével implementáltam. A WorkManager egy megbízható, háttérfeladatok kezelésére szolgáló Android komponens, amely biztosítja, hogy a feladatok akkor is végrehajtsódjanak, ha az alkalmazás nincs aktívan használatban vagy az eszköz újraindult.[29]

2.4.5 Függőséginjektálás

Az alkalmazásban a függőségek injektálását a Hilt segítségével valósítottam meg, amely a Google által fejlesztett, modern függőséginjekciós könyvtár. A Hilt célja, hogy leegyszerűsítse a függőségek kezelését és elosztását az alkalmazás különböző komponensei között, miközben csökkenti a kód komplexitását és javítja annak tesztelhetőségét. A Hilt automatikusan kezeli a komponensek élettartamát és a közöttük lévő kapcsolatokat, így biztosítva, hogy minden szükséges objektum időben és megfelelően kerüljön injektálásra, anélkül, hogy manuálisan kellene konfigurálni.[30]

2.4.6 Tesztelés

Az alkalmazás funkcióinak teszteléséhez a JUnit 4[31] keretrendszert alkalmaztam. A különböző függőségek izolálásához a MockK[32] könyvtárat, az

aszinkron folyamatok teszteléséhez pedig a Kotlin Coroutines Test[33] könyvtárat használtam fel.

A felhasználói felületek teszteléséhez szükségem volt továbbá a JUnit 4 Compose UI Test[34] kiegészítő könyvtárra is. Ebben a platformfüggő környezetben a függőségek izolálását a MockK Android[35] könyvtár, a függőségek injektálását és kezelését pedig a Hilt Test[36] könyvtár segítségével valósítottam meg.

2.4.7 Folyamatos integráció

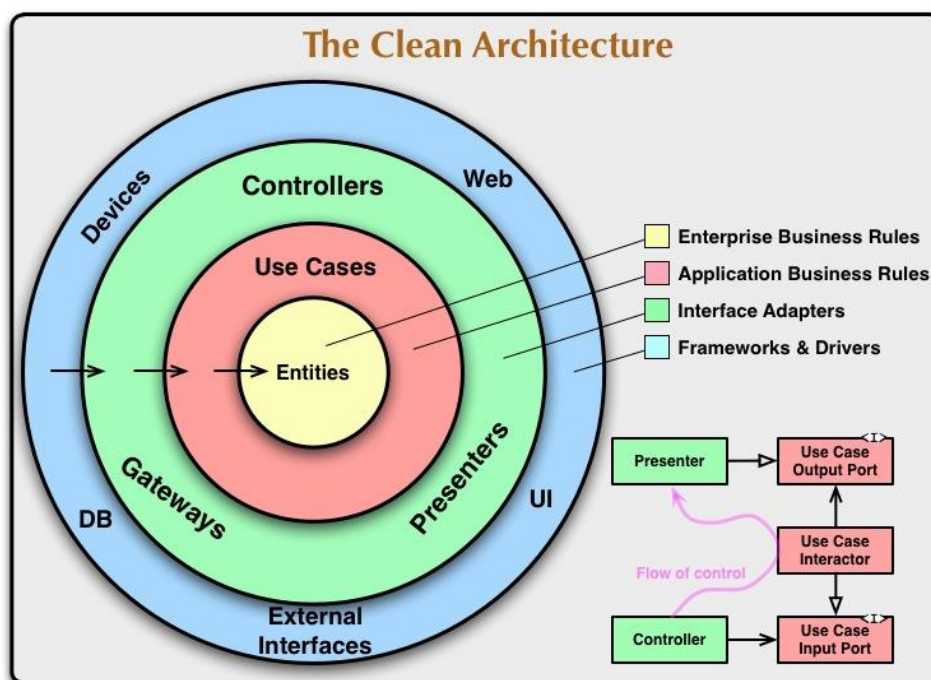
A projekt verziókezelését a GitHub[37] segítségével végeztem, amelyet egyetemi tanulmányaim során számos különböző projektben használtam, így kézenfekvő választás volt. Ami eltért a korábbi tapasztalataimtól, hogy a projekthez készítettem egy folyamatos integrációt (continuous integration, CI)[38] biztosító automatizált feldolgozási láncot (pipeline) a GitHub Actions[39] segítségével. Ez lehetővé teszi automatizált munkafolyamatok (workflow) létrehozását, például a kód fordítását, tesztelését és telepítését, így biztosítva, hogy minden módosítás gyorsan és megbízhatóan integrálódjon a fő ágba. A megoldás jelentősen növeli a fejlesztés hatékonyságát, és csökkenti a hibák kockázatát a folyamat során.

3 Architektúra

Az alkalmazás architektúrája az Android fejlesztési irányelveinek megfelelően a Clean Architecture és az MVVM (Model-View-ViewModel) mintákra épül.

3.1 Clean Architecture

A Clean Architecture[40] lehetővé teszi az alkalmazás logikai rétegeinek elkülönítését, így biztosítva, hogy az egyes rétegek, mint a felhasználói felület, az üzleti logika és az adatkezelés, önállóan fejleszthetők, tesztelhetők és cserélhetők legyenek anélkül, hogy az alkalmazás többi része jelentős módosítást igényelne. Ez a megközelítés megkönnyíti az alkalmazás karbantartását és továbbfejlesztését, továbbá növeli a kód átláthatóságát és az egyes komponensek újrafelhasználhatóságát.



1. ábra: Clean Architektúra felépítése[41]

A Clean Architecture tipikusan négy fő rétegre bontható:

1. **Entities** (Vállalati üzleti szabályok): A legbelső réteg, amely az alkalmazás alapvető üzleti logikáját és adatszerkezeteit tartalmazza. Az ide tartozó entitások függetlenek minden külső technológiától, így a legstabilabb, legkevésbé változó részek közé tartoznak, ezáltal bárhol újrahasznosíthatók.

2. **Use Cases** (Alkalmazás-specifikus üzleti logika): Ez a réteg határozza meg az alkalmazás specifikus működését. A use case-ek a felhasználói igényeket szolgálják ki, irányítva az adatáramlást és a működést. A konkrét alkalmazásra szabottak, ugyanakkor függetlenek az olyan részletektől, mint az adatbázis vagy a felhasználói felület.
3. **Interface Adapters** (Felületi adapterek): Ez a réteg olyan adaptereket tartalmaz, amelyek az adatokat a use case-ek és entitások számára legmegfelelőbb formátumból a külső rendszerek, például az adatbázis vagy a felhasználói felület, számára legmegfelelőbb formátumba alakítják. Ide tartoznak többek között a kontrollerek és prezenterek.
4. **Frameworks & Drivers** (Keretrendszerek és külső eszközök): Ez a legkülső réteg, amely tipikusan a konkrét technológiai megvalósításokat tartalmazza: adatbázis, felhasználói felület, további külső eszközök. Itt főként olyan kód található, amely az interfészeket hívja vagy szolgálja ki. Ezek a részletek gyakran változnak, ezért kerülnek a legkülső rétegbe, hogy ne befolyásolják a belső logikát. A cél, hogy az alkalmazásmag független maradjon ezektől a változó komponensektől.

Az adatáramlás kívülről befelé történik, míg az irányítás (flow of control) belülről kifelé halad. Ez biztosítja, hogy a belső rétegek soha ne fűggenek a külsőktől, ezzel támogatva a tesztelhetőséget, újrafelhasználhatóságot és a technológiai függetlenséget.

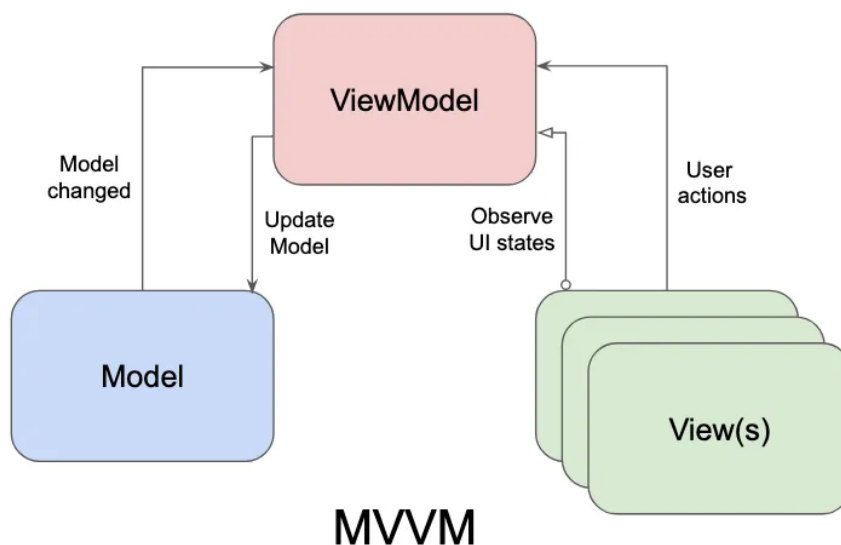
3.2 Model-View-ViewModel

A Model-View-ViewModel (MVVM)[42] egy, az Android alkalmazások fejlesztése során gyakran használt szoftver architektúrális minta, amely kifejezetten az adatvezérelt felhasználói felületekhez készült. Az MVVM fő előnye, hogy a felhasználói felület logikáját elválasztja a mögöttes adatmodelltől, így könnyebbé válik a kód tesztelése és karbantartása.

Az MVVM az alábbi három komponensből áll:

1. **Model**: Az adatok, valamint üzleti logika megvalósítását tartalmazza. Feladatai közé tartoznak például az adatok betöltése, feldolgozása és tárolása. A Model mind a View-től, mind a ViewModel-től független.

2. **View:** Az alkalmazás felhasználói felületének megvalósítása. Feladata az adatok megjelenítése a felhasználó számára, valamint a felhasználói interakciók, például gombnyomás vagy szövegbevitel továbbítása a ViewModel felé. A View önmagában nem tartalmaz logikát, csak a megjelenítéssel foglalkozik.
3. **ViewModel:** Összeköti a View-t és a Model-t. Lekéri az adatokat a Model-től, előkészíti őket megjelenítésre, majd megfigyelhető formában, például állapot formájában elérhetővé teszi a View számára. A View ezen állapot megfigyelésével képes automatikusan frissíteni a felhasználói felület tartalmát, amikor az adatok megváltoznak. A ViewModel feladata továbbá a felhasználói műveletek kezelése, illetve szükség esetén a Model állapotának módosítása. Fontos, hogy a ViewModel nem tartalmaz referenciát a View-ra, így az könnyebben tesztelhető.



2. ábra: MVVM architektúra felépítése[43]

3.3 Az alkalmazás felépítése

Az architektúrális minták ismertetése után a következőkben szeretném bemutatni az általam implementált alkalmazás felépítését, az adatrétegtől a felhasználói felület felé haladva.

3.3.1 Adatforrások

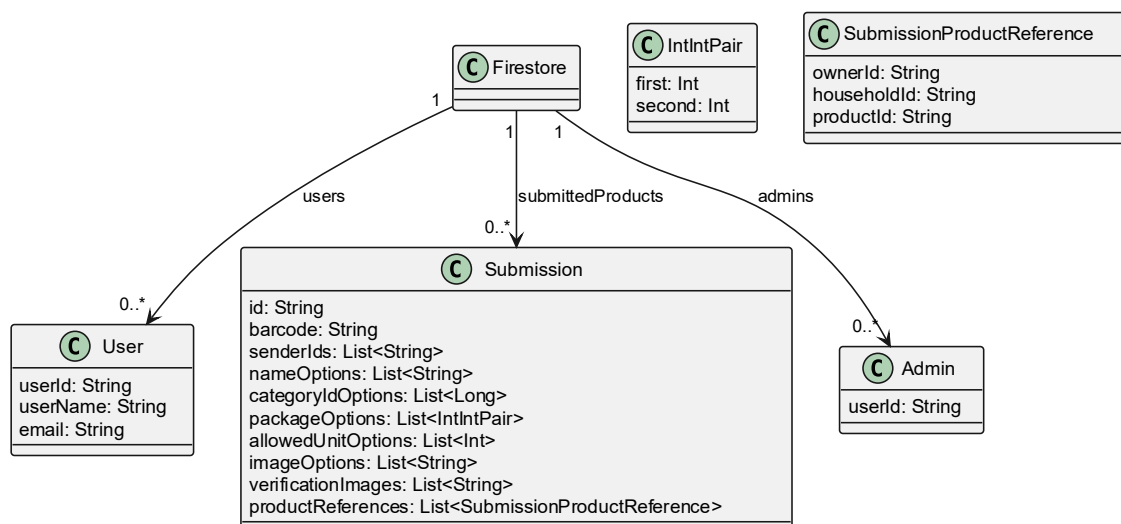
Az alkalmazás két fő adatforrása a Firebase, illetve a Supabase, melyek egyaránt felhő alapú szolgáltatások. Ezen megoldások mellett az offline megjelenítés megvalósításához szükségem volt egy lokális cache implementálására, melyhez a Room könyvtárat használtam.

3.3.1.1 Firestore

A Firestore adatbázis felel a felhasználókhöz kapcsolódó adatok tárolásáért, beleértve a háztartásokat, valamint az azokhoz tartozó bevásárlólistákat, készletelemeket és termékeket. Ezen adatok mellett itt kerülnek eltárolásra az alkalmazáson belül kiküldött értesítések, a készletelemek lejáratához kapcsolódó emlékeztetők, valamint a termék-hitelesítési folyamat során létrehozott termékbeküldések is.

Az alkalmazás bizonyos funkciói, például a termékbeküldések kezelése kizárólag adminisztrátori jogosultsággal érhetők el. Az ilyen jogosultsággal rendelkező felhasználók azonosítói szintén a Firestore-ban, egy külön adatkollekcióban vannak eltárolva.

Mivel a felhasználókezelés szintén a Firebase-ben került megvalósításra, kézenfekvőnek tartottam ezt a megoldást használni, hiszen így közvetlenül és biztonságosan szabályozható a felhasználók adathozzáférése.



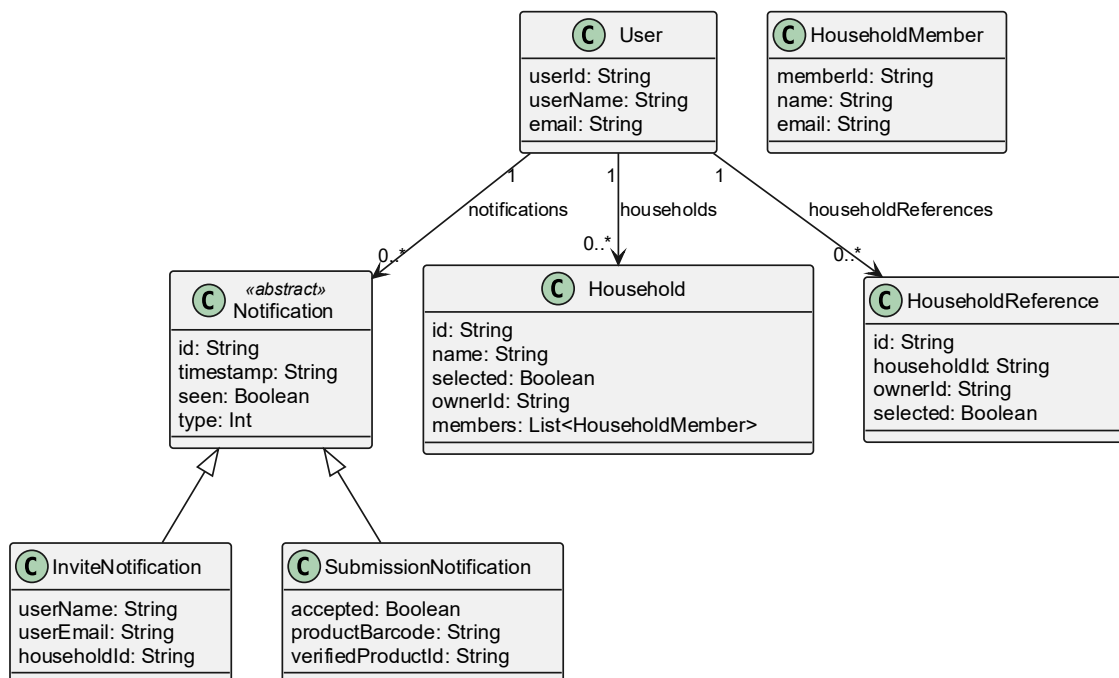
3. ábra: A Firestore adatbázis séma gyökere

A Firebase adatbázis szerkezetének kiindulópontját a Firestore szolgáltatás jelenti, ezen keresztül érhetők el az adatbázisban tárolt gyűjtemények (collection-ök),

ahogyan ez a 3. ábra is szemlélteti. A jelen alkalmazás esetében három fő gyűjtemény található ezen a szinten: a felhasználók adatai, a termékbeküldések, valamint az adminisztrátori jogosultságokkal rendelkező felhasználók azonosítói.

A termékbeküldések gyűjteményben a hitelesítésre beküldött termékek dokumentumai a termékekhez tartozó vonalkódok alapján vannak csoportosítva. Egy adott vonalkódhoz mindig egy dokumentum tartozik, így, ha egy felhasználótól újabb beküldés érkezik ugyanahhoz a termékhez, akkor nem jön létre új dokumentum, hanem a meglévő bővül az új adatokkal. Emiatt a termék tulajdonságai (például a név, kategória, a kiserelés adatai vagy a termék képe) listák formájában kerülnek tárolásra, így a rendszer több felhasználói beküldést is egységesen képes kezelni. A hitelesítési folyamat során az adminisztrátor ezekben a listákban tárolt opciókból választja ki a végleges, jóváhagyott adatokat.

A dokumentum további mezői, például a beküldők azonosítói (*senderIds*), az ellenőrzőfotók (*verificationImages*), valamint a más adatkollekciókra mutató hivatkozások (*SubmissionProductReference*), mind a termék-hitelesítési folyamat visszakövethetőségét szolgálják.



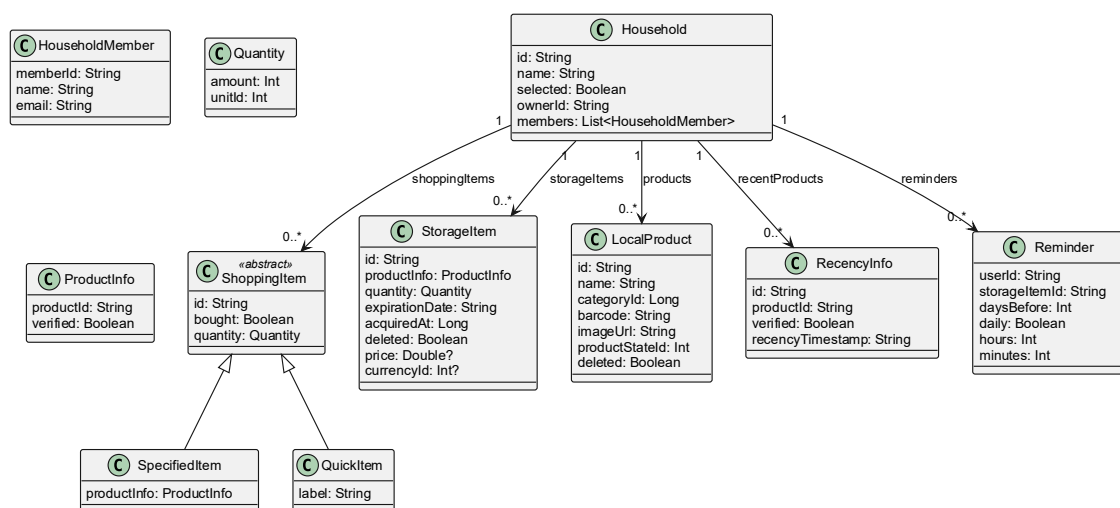
4. ábra: A felhasználókön keresztül közvetlenül elérhető gyűjtemények

A felhasználói dokumentumokon belül három adatkollekció érhető el: a háztartások (*Household*), a háztartásreferenciák (*HouseholdReference*), valamint az

értesítések (*Notification*), ezen összefüggéseket a 4. ábra szemlélteti. A háztartások és a háztartásreferenciák szorosan összekapcsolódnak, ugyanakkor külön szerepet töltenek be. A háztartásokhoz tartozó kollekció a felhasználó által létrehozott, saját háztartások adatait tartalmazza, beleértve a háztartás nevét, tulajdonosát, valamint a tagok listáját. A háztartásreferenciák ezzel szemben azon háztartásokra vonatkoznak, amelyeket nem a felhasználó hozott létre, azonban meghívón keresztül csatlakozott hozzájuk. Ennek köszönhetően a felhasználó könnyedén válthat az általa kezelt és a hozzá kapcsolódó, mások által létrehozott háztartások között.

Az értesítésekhez tartozó kollekció az alkalmazáson belüli értesítéseket tartalmazza. Két konkrét értesítéstípus létezik, a meghívóértesítések (*InviteNotification*), valamint a termékbeküldésekhez kapcsolódó értesítések (*SubmissionNotification*).

A meghívóértesítések tartalmazzák a küldő felhasználó nevét, e-mail címét, és a kapcsolódó háztartás azonosítóját. A termékbeküldések értesítései ezzel szemben a beküldött termék vonalkódját, a hitelesítési kérelem elfogadásának állapotát, illetve elfogadás esetén a hitelesített termék azonosítóját tartalmazzák. Ezek az értesítések teszik lehetővé, hogy a felhasználók meghívókat tudjanak küldeni egymásnak, illetve, hogy értesüljenek a termék-hitelesítési folyamat eredményeiről.



5. ábra: A háztartások által tartalmazott gyűjtemények

Az 5. ábra a háztartások adatstruktúráját mutatja be, vagyis azt, hogy egy adott háztartás milyen típusú adatkollekciókat tartalmaz a Firestore-on belül. Egy háztartás dokumentuma több jól elkülönülő adatszerkezetet foglal magában, amelyek a készletek, bevásárlólisták, emlékeztetők és termékadatok kezelésének alapját képezik.

A háztartáshoz tartozó tagok listáját a *HouseholdMember* objektumok reprezentálják, amelyek a felhasználó azonosítóját, nevét és e-mail címét tárolják. Ezek az adatok teszik lehetővé, hogy a felhasználók közösen tudják használni a háztartásokat, azaz több felhasználó egyszerre férhessen hozzá ugyanahhoz a készlethez vagy bevásárlólistához.

A készletelemek a *StorageItem* kollekción keresztül érhetők el. Minden elem egy konkrét termékre hivatkozik a *ProductInfo* struktúra segítségével, amely tartalmazza a termék azonosítóját, illetve a termék hitelesítettségi állapotát. A készletelem rögzíti többek között a megvásárolt mennyiséget, a lejárat dátumot, a készletelem árát és annak pénznemét, valamint a *deleted* mezőjén keresztül támogatja a logikai törlés kezelését is. Ez utóbbi lehetővé teszi, hogy az aktívan használt készletelemek eltávolításukat követően is megtekinthetők legyenek a készletelőzményekben.

A bevásárlólista elemei a *ShoppingItem* absztrakt osztályból származó struktúrákban jelennek meg. Ebben a kollekcióban két különböző típusú tételek jelenhetnek meg, melyek szintén szerepelnek az ábrán. A *SpecifiedItem* egy konkrét termékre hivatkozik (*ProductInfo*), a *QuickItem*, pedig csak egy szöveges címkét tárol. Utóbbi olyan listatételek rögzítésére szolgál, amelyekhez a listára való felkerüléskor még nem tartozik termékhivatkozás. Ezzel az adatstruktúrával rugalmasan kezelhetők a felhasználók által felvett, gyors vagy részletes bevásárlólista-elemek is.

A háztartáshoz kapcsolódóan a felhasználók saját termékeket is létrehozhatnak. Ezek a *LocalProduct* dokumentumok tartalmazzák a termék nevét, vonalkódját, kategóriájának azonosítóját, a termék képének elérési útvonalát, valamint a termék státuszát. A *deleted* mező itt is a logikai törlés kezelésére szolgál, amely szintén a készletelőzmények miatt szükséges.

A *RecencyInfo* kollekció a termékelőzmények tárolására szolgál. Ennek célja, hogy tárolja az előre meghatározott darabszámú, legutóbb hivatkozott termékeket. Az itt tárolt adatok alapján kerül megjelenítésre a legutóbb használt termékek listája.

Végezetül a *Reminder* kollekció tartalmazza a lejárat dátumokhoz kapcsolódó emlékeztetőket. Ezek a dokumentumok meghatározzák, hogy az adott készletelem lejárat előtt hány nappal, illetve milyen időpontban kerüljön kiküldésre értesítés a

felhasználó számára. Továbbá megkülönböztethetők egyszeri és ismétlődő értesítések, melynek értéke a *daily* mezőben van eltárolva.

3.3.1.2 Supabase Storage

A Supabase szolgáltatás kétféle adattárolást is végez. Egyrészt mivel a Firebase már nem biztosít ingyenes lehetőséget erőforrásfájlok, például képek tárolására, az alkalmazásban szereplő termékekhez tartozó képeket a Supabase Storage segítségével tároltam el. Bár ezen képek egy része szintén felhasználói fiókokhoz kötődik, az elérésük publikusan, az egyedi azonosítójuk alapján történik. Mivel a képfeltöltés opcionális, és a termékfotók nem minősülnek személyes adatnak, továbbá a képek feltöltése kizárólag a felhasználók felelősségére történik, mivel Androidon a képek elérése és a kamera használat is engedélyhez van kötve, ezért ezt nem gondolom adatvédelmi kockázatnak.

A képek tárolására három külön tárolót hoztam létre. Az első tároló a privát termékekhez kapcsolódó képeket tartalmazza. A képek háztartásonként vannak mappákba rendezve, minden mappa neve a háztartás azonosítója, a benne található képek neve pedig a hozzájuk tartozó termék azonosítója. Ez a struktúra megkönnyíti a képek kezelését: új kép feltöltésekor az elérési útvonal változatlan marad, törlés esetén pedig egyszerűen beazonosítható a kép, illetve egy teljes háztartás törlésekor maga a mappa is könnyen megtalálható.

A második tároló a hitelesítésre beküldött termékekhez tartozó hitelesítő képeket tartalmazza. Ezek olyan képek, amelyeken egyértelműen látható, hogy a beküldött termék és a hozzá tartozó vonalkód valóban összetartoznak, vagyis a vonalkód a termék csomagolásán szerepel. Ennek felépítése megegyezik a privát termékek képeit tartalmazó tárolóval, azaz itt is háztartásonkénti mappastruktúra biztosítja a könnyű kezelhetőséget.

A harmadik tárolóban a minden felhasználó számára elérhető, hitelesített termékek képei találhatók. Ebben az esetben nincs mappastruktúra, minden kép közvetlenül a tároló gyökerében helyezkedik el, a képek neve pedig a megszokott módon a hozzájuk tartozó termék azonosítója.

3.3.1.3 Supabase Database

A Supabase által biztosított másik adatelérési forma a hitelesített termékek adatainak tárolása, ehhez a Supabase Database relációs adatbázisát használtam fel. Az alkalmazás egyik alapkoncepciója, hogy termékadatbázis a felhasználók által történő

termék beküldésekkel bővül. Mivel várhatóan idővel több ezer termék is bekerülhet az adatbázisba, fontosnak tartottam egy olyan tárolási megoldást választani, amely szerveroldalon biztosít hatékony szűrési és keresési lehetőségeket, így minimalizálva a hálózati forgalmat. A Firestore ezeknek az elvárásoknak nem felelt meg, a Supabase Database viszont igen, ezért az utóbbit választottam.

A termékekhez kapcsolódóan két adatbázis táblát hoztam létre. A *verified_products* tábla a közös termékek tulajdonságait, a *price_history* tábla pedig a termékárhoz kapcsolódó bejegyzéseket tartalmazza.

A közös termékekhez eltárolásra kerül többek között a termék neve, vonalkódja, kategóriája, valamint a kiszerelés mérete és az ahhoz tartozó mértékegység, melyek elsősorban a termék megjelenítésekor kapnak szerepet. Az attribútumok között szerepel továbbá a *normalized_name*, amely a terméknevé szabványosított formáját tartalmazza a keresések hatékonyabbá tétele érdekében, illetve az *image_url*, amely a termékhez tartozó kép elérési útvonalát rögzíti. Ez utóbbi két érték a termék feltöltését követően, egy adatbázis-trigger segítségével kerül automatikusan kiszámításra. A kép elérési útvonalának szerveroldali előállításához kiemelten fontos a képek tárolásánál korábban említett mappastruktúra és nevezéktan.

A *price_history* tábla a termékekhez kapcsolódó áradatokat rögzíti. Minden bejegyzés egy adott termékhez (*product_id*), illetve egy készletelemhez (*storage_item_id*) kapcsolódik. Az árakhoz rögzítésre kerül a létrehozás és frissítés időpontja is (*created_at*, *updated_at*), melyek lehetővé teszik az árak időbeli változásának követését.

Bár alapvetően elegendő lenne csak a létrehozás időpontját tárolni, előfordulhat, hogy a felhasználó módosítja egy korábban rögzített készletelem árát (például elírás miatt). Ilyenkor a frissítés idejének megváltozása biztosítja, hogy az adott bejegyzés újra relevánssá váljon. Ennek köszönhetően az alkalmazásban megjelenített mediánárak mindig a legfrissebb felhasználói adatok alapján kerülnek kiszámításra.

verified_products	
id	uuid
name	text
barcode	text
category_id	int8
image_url	text
normalized_name	text
quantity_unit_ids	_int4
package_size	int4
package_unit_id	int4

price_history	
# id	int8
product_id	uuid
price	float8
created_at	timestampz
storage_item_id	text
updated_at	timestampz

6. ábra: Supabase adatbázis séma.

3.3.1.4 Lokális cache

Az alkalmazás offline működésének biztosításához elengedhetetlen volt a lokális cache alkalmazása. A Firestore automatikusan támogatja ezt a funkciót, ezzel szemben a Supabase Database esetén a lekérdezett adatokat szükséges manuálisan letárolni. Ehhez az Android alkalmazások fejlesztése során széles körben használt Room könyvtárat használtam. A Room, akár csak a Supabase Database, relációs adatbázis létrehozását teszi lehetővé, megkönnyítve ezzel az adatok közti konverziót. A termékek helyi tárolása ugyanakkor nem lenne előnyös az alkalmazás mérete és háttértárhasználata szempontjából, így offline módban a képek helyett az adott termék kategóriáját reprezentáló helyettesítő kép jelenik meg.

3.3.2 Adatkezelés

Az adatkezelési réteg feladata, hogy megteremtse a kapcsolatot az üzleti logikai réteg, valamint az adatforrások között, ezáltal elérhetővé tegye az alkalmazás számára szükséges adatokat. Két főbb alrétegből áll: DataSource és Repository.

A DataSource osztályok az adatforrásokkal való közvetlen kommunikációt valósítják meg. Feladatuk az adatok manipulálása és az adatelérés biztosítása a Repository-k számára. Az implementáció teljes mértékben a használt technológiától függ, ugyanakkor a kapcsolat a Repository-k felé interfészen keresztül történik, ezzel megőrizve a modularitást.

Az alkalmazásban minden DataSource-hoz tartozik egy Repository osztály, amely a DataSource által nyújtott funkcionalitást elérhetővé teszi az üzleti logikai réteg számára. A Repository-k elvégzik az üzleti logikai réteg és az adatkezelési réteg által használt adatformátumok közötti konverziót, validálják a beérkező adatokat, metódushívásokat, biztosítják a kontextust és az egyéb szükséges paramétereket a DataSource-ok számára. Ezen túlmenően egyes Repository-k más Repository-kat is felhasználhatnak a komplex feladatok megvalósításához. Hasonlóan a DataSource réteghez, a Repository-k által kínált metódusok is interfészen keresztül érhetők el a felsőbb rétegek számára. Ez a réteg már független az adott adatforrás típusától, az üzleti logikában definiált modell osztályokat használja fel.

3.3.3 Üzleti logika

Az üzleti logikai réteg felelős az alkalmazás működését meghatározó folyamatok megvalósításáért. Itt található az alkalmazás által használt modellobjektumok osztályai, valamint a különböző cselekvéseket megvalósító use case-ek. Ez a réteg biztosítja a kapcsolatot a megjelenítési és az adatkezelési réteg között.

Minden use case egy-egy cselekvést, vagyis az alkalmazás egy konkrét funkcióját reprezentálja, és annak működését írja le az üzleti logika szintjén. Ilyen cselekvés lehet például egy készletelem létrehozása, egy háztartás meghívó elküldése vagy egy termék hitelesítésre történő beküldése. A megjelenítési rétegből érkező felhasználói műveletek ezeken a use case-eken keresztül valósulnak meg, így a ViewModel-ek nem tartalmazznak üzleti logikai implementációkat.

A modell osztályok az üzleti logika által kezelt adatstruktúrákat írják le. Felépítésük gyakran hasonlít a Firestore-ban tárolt dokumentumok szerkezetére, ugyanakkor előfordulhatnak kisebb eltérések. Ilyen lehet például a Firestore-specifikus adattípusok alkalmazásszintű reprezentációja, illetve tartalmazhatnak olyan tulajdonságokat vagy számított értékeket, amelyek adatbázisban nem kerülnek eltárolásra, de az alkalmazás működése során szükségesek. A modell osztályok egyik nagy előnye, hogy a felhasználói felület sok esetben közvetlen adatforrásként használhatja őket, így nincs szükség további átalakítási lépésekre az adatok megjelenítése előtt.

Összességében az üzleti logikai réteg definiálja az alkalmazás működését, eközben pedig világosan elhatárolja egymástól a megjelenítési és az adatkezelési réteg felelősségi köreit.

3.3.4 Felhasználói felület

Az alkalmazás felhasználói felülete Jetpack Compose alapú, ahol az egyes képernyők funkcionalitását a *Screen* utótaggal ellátott Composable függvények valósítják meg. Ezek a Composable-ök felelnek a megjelenítésért, a vizuális elemek elrendezéséért, továbbítják a felhasználói interakciókat a ViewModel-ek felé, valamint a ViewModel-ben tárolt állapot alapján frissítik a felületen megjelenő tartalmat.

Minden képernyőhöz tartozik legalább egy ViewModel, amely az MVVM architektúra elveinek megfelelően biztosítja a felület számára szükséges adatokat és állapotokat. A ViewModel nemcsak a megjelenítendő információk előkészítéséért felel, hanem kezeli a felhasználói műveleteket is, például gombnyomásokat, szövegbevitelt, vagy elemkiválasztásokat. Ezen kívül validációt is végez a felhasználói bemeneteken, és gondoskodik az esetleges hibakezelésről is.

A ViewModel összeköttetést biztosít a megjelenítési réteg és az üzleti logika között: a felhasználói interakciókból származó adatokat és eseményeket továbbítja a megfelelő use case-ek felé, majd az onnan visszaérkező eredmények alapján frissíti a tartalmazott belső állapotát. Ennek köszönhetően a felhasználói felület rétege logikailag elkülönül a háttérrendszertől, ami könnyebb tesztelhetőséget és karbantarthatóságot biztosít, továbbá elősegíti az újrafelhasználhatóságot.

A felhasználói felületi réteghez szorosan kapcsolódik az alkalmazás képernyői közötti navigáció is. A navigáció során használt útvonalakat a *Screen* interfész és az őt megvalósító osztályok definiálják. A navigációs logika két különálló navigációs gráfból áll: a külső navigációs gráf kezeli az autentikációs folyamat képernyőit, a bejelentkezés és a regisztráció, valamint az alkalmazás belső működése közötti átmenetet, a belső gráf pedig az autentikációt követően az alkalmazáson belüli navigációt biztosítja.

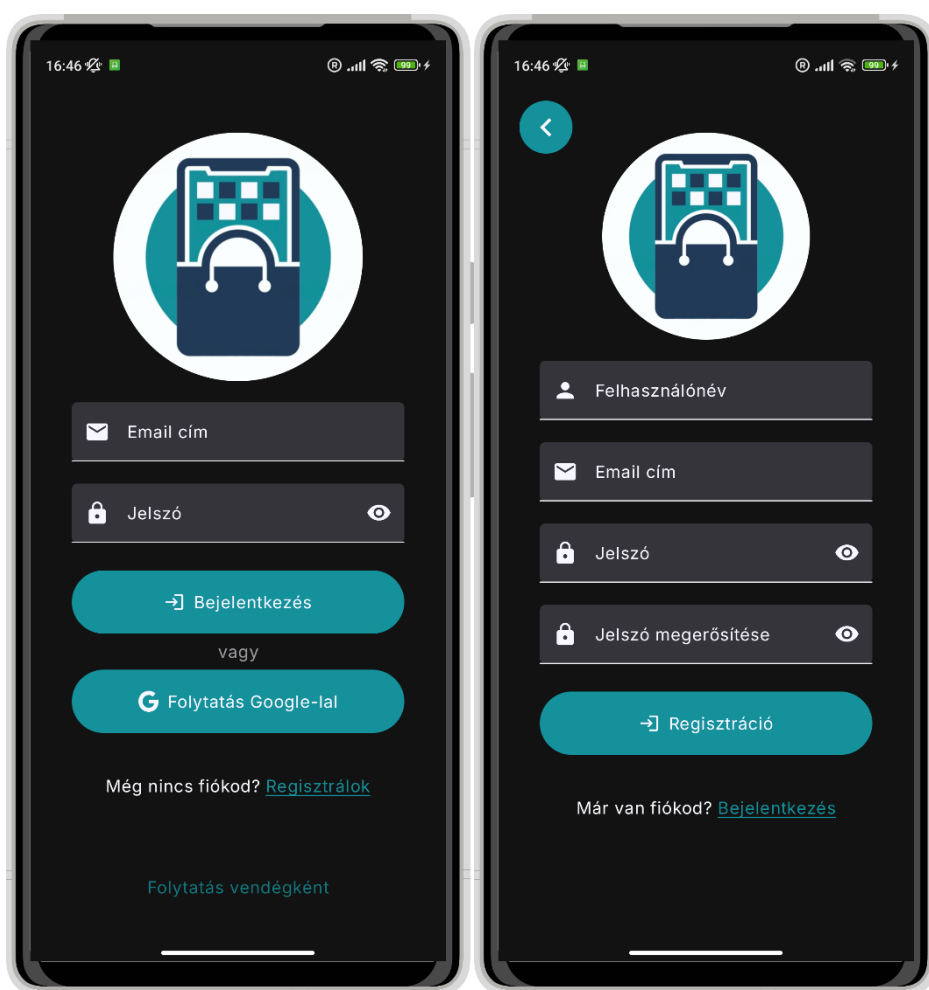
4 Képernyők, funkciók

Ebben a fejezetben bemutatom az alkalmazás különböző funkcióit és felhasználói felületének egyes képernyőit, a szemléltetés érdekében képernyőképekkel illusztrálva.

4.1 Felhasználókezelés

Az alkalmazás használata felhasználói fiókhoz kötött, ami lehetővé teszi, hogy a felhasználók adatai több eszközön keresztül is szinkronizáltan elérhetők legyenek. A hitelesítés emellett személyre szabott élményt nyújt, valamint biztosítja az adatok elkülönítését és védelmét.

4.1.1 Bejelentkezés, regisztráció

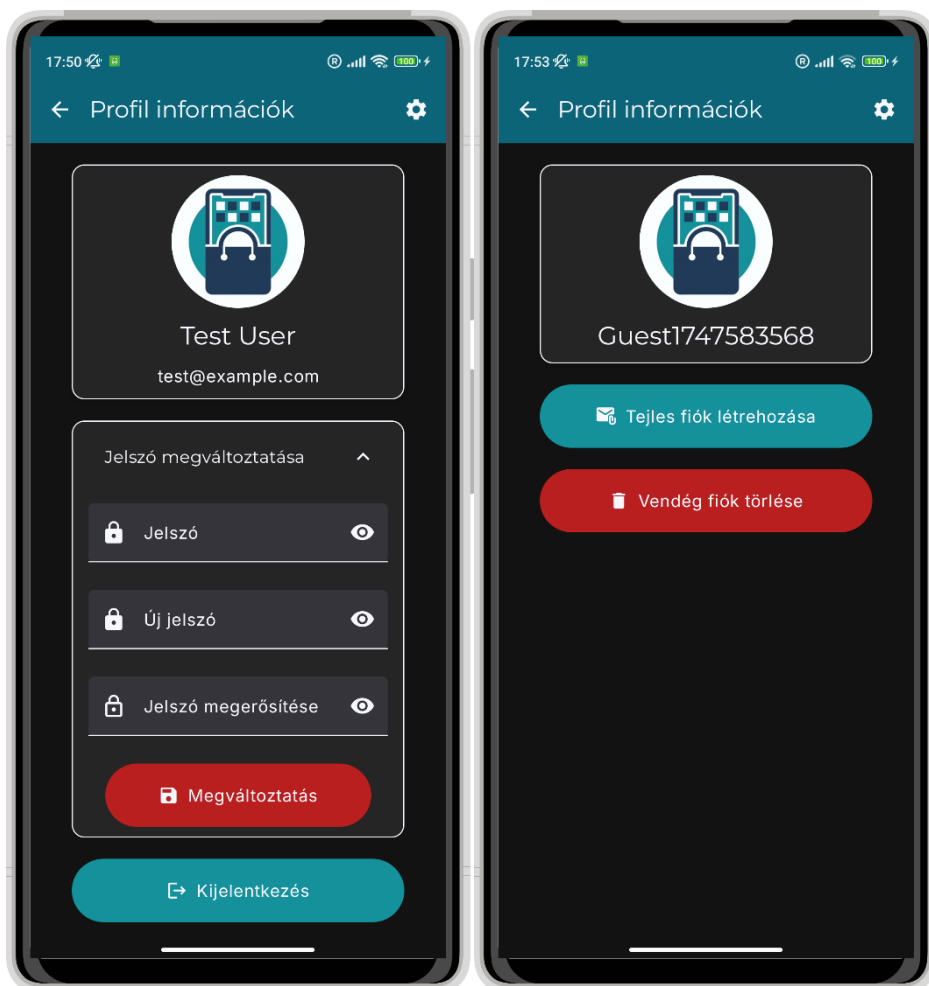


7. ábra: Bejelentkezés és regisztrációs képernyők.

Az alkalmazás első indításakor a bejelentkezési felület jelenik meg, ahol a felhasználó háromféle hitelesítési lehetőség közül választhat: bejelentkezés e-mail cím és jelszó megadásával, Google-fiók használatával (Google Auth), illetve vendégként történő bejelentkezés. Utóbbi lehetőség különösen hasznos azok számára, akik először szeretnék kipróbálni az alkalmazást anélkül, hogy azonnal fiókot hoznának létre.

Amennyiben a felhasználó még nem rendelkezik regisztrált fiókkal, a *Regisztrálok* gomb segítségével a regisztrációs képernyőre navigálhat, ahol tetszőleges felhasználónév, érvényes e-mail cím és jelszó megadásával hozhat létre új fiókot. Azon felhasználók, akik korábban már bejelentkeztek, az alkalmazás későbbi megnyitásakor automatikusan bejelentkeztetésre kerülnek.

4.1.2 Profilképernyő



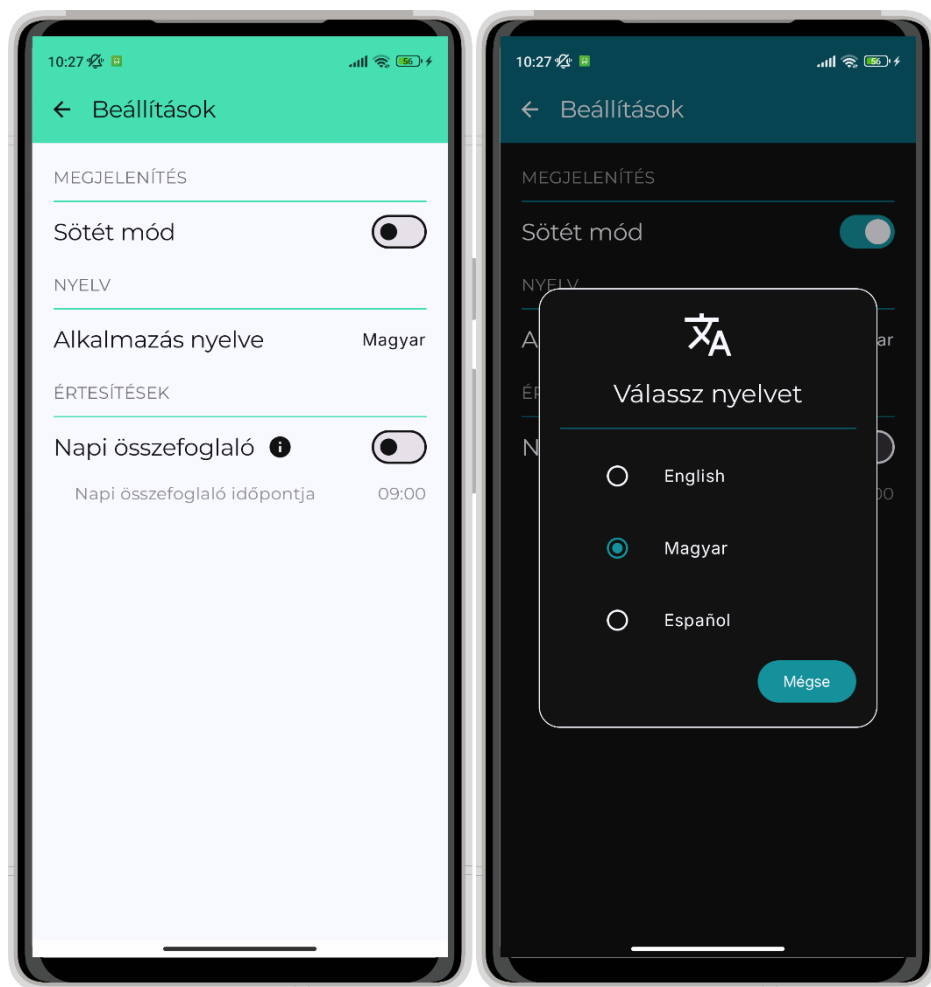
8. ábra: Profilképernyő teljes, illetve vendég fiók esetén.

A felhasználókezeléshez szorosan kapcsolódik a profilképernyő is, amely a bejelentkezést követően a jobb felső sarokban található *személy* ikonnal érhető el. A

profilnézeten a felhasználó adatai jelennek meg, valamint innen érhetők el a jelszó módosítására és a kijelentkezésre szolgáló funkciók is.

A vendégként bejelentkezett felhasználók számára a profilképernyő lehetőséget biztosít a vendégfiók törlésére, valamint teljes fiók létrehozására. Utóbbi esetben a felhasználó e-mail cím és jelszó megadásával regisztrálhat, vagy választhatja a Google-fiókkal történő hitelesítést is. A vendégfiók teljes fiókká alakítása során az alkalmazás automatikusan összekapcsolja a korábban használt adatokat az újonnan létrehozott fiókkal, így a felhasználó adatvesztés nélkül folytathatja az alkalmazás használatát.

4.1.3 Beállítások



9. ábra: Beállítások képernyő világos, illetve sötét módban.

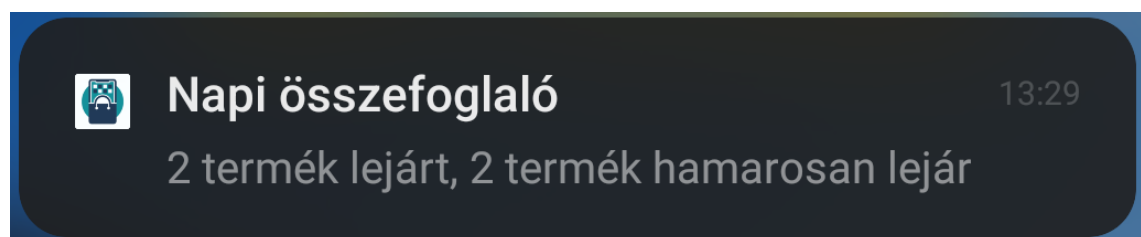
A profilnézet jobb felső sarkában található *fogaskerék* ikonnal érhető el a beállítások képernyő. Itt megváltoztatható az alkalmazás témája (világos vagy sötét mód), valamint kiválasztható a felhasználói felület nyelve. Az alkalmazás három nyelvet támogat: magyar, angol és spanyol.

4.1.3.1 Napi összefoglaló

A napi összefoglaló beállítási opció lehetővé teszi, hogy a felhasználók naponta értesítést kapjanak a készleteik állapotáról. Az értesítés tartalmazza, hogy hány olyan készletelem található a háztartásban, amely már lejárt, valamint azt is, hogy hány elem fog lejárni a következő három napon belül. A funkció célja, hogy a felhasználók akkor is naprakészen követhessék készleteik alakulását, amikor éppen nem használják aktívan az alkalmazást.

A beállításhoz tartozik egy időválasztó is, amelynek segítségével a felhasználó megadhatja, hogy hány órákor szeretné megkapni a napi összefoglalókat. Mivel az értesítések nem percre pontosan a beállított időpontban kerülnek kiküldésre, az időválasztó 15 perces pontossággal állítható. A beállítás mellett található információs ikon megnyomásával a felhasználó egy rövid magyarázatot is elolvashat az értesítések működéséről, amely összefoglalja a korábban ismertetett főbb tudnivalókat.

A napi összefoglalók push értesítések formájában kerülnek kiküldésre, melyek formátuma a következő ábrán látható.

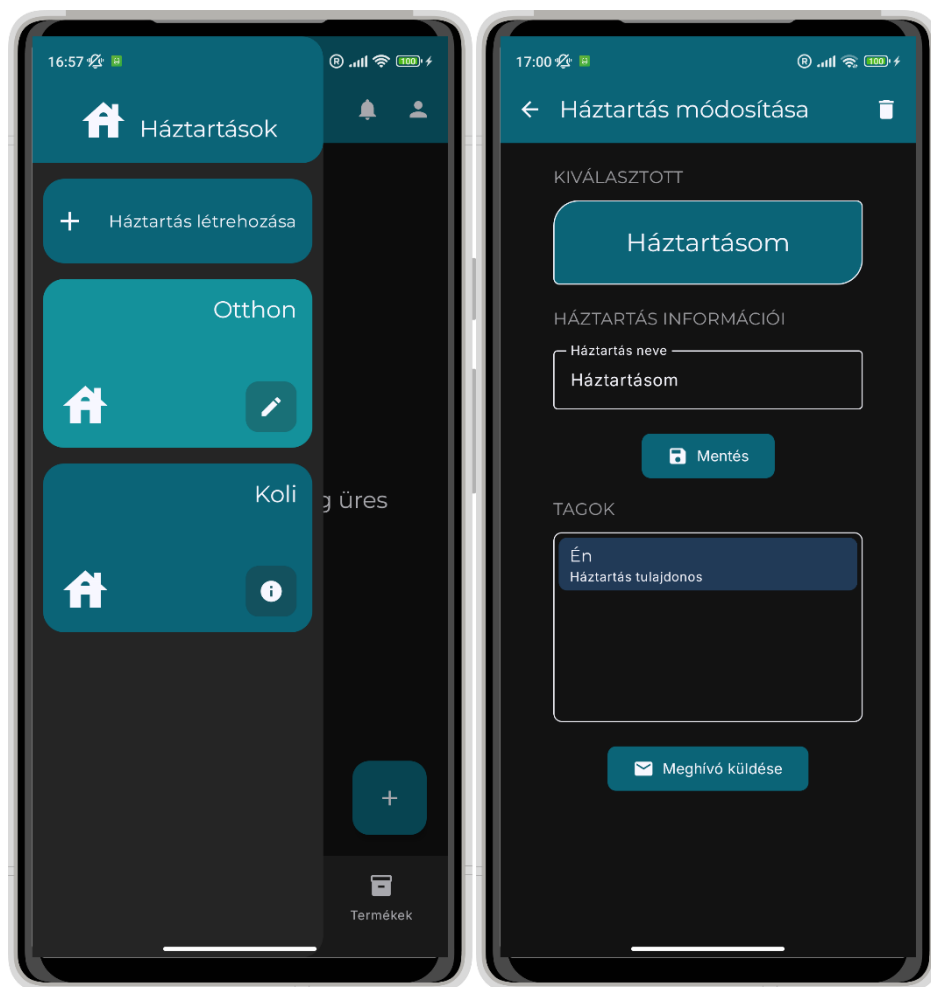


10. ábra: Napi összefoglaló értesítés

4.2 Háztartások

Az alkalmazás működésének alapját a háztartások képezik. Egy háztartás magában foglal egy bevásárlólistát, a készleteket, valamint az adott háztartáshoz tartozó, felhasználók által létrehozott termékeket. A felhasználók tetszőleges számú háztartást hozhatnak létre, melyek a bal oldalon található navigációs menüből érhetők el. A módosítás képernyőn lehetőség van módosítani a háztartás nevét, illetve törölhető a teljes háztartás. Törlés esetén minden tartalmazott adat elveszik, ezért a műveletet egy felugró dialógusablakban szükséges megerősíteni.

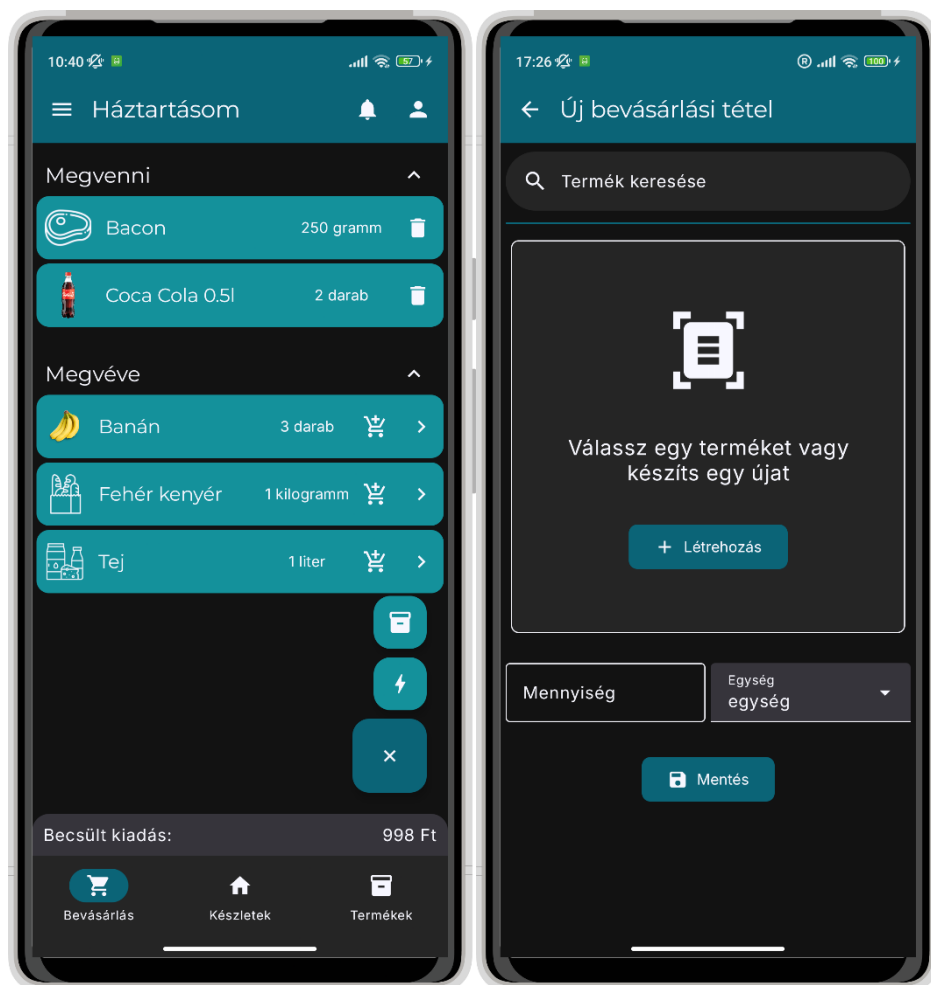
A háztartások közös használatát támogatja a meghívó küldése funkció, amely lehetővé teszi, hogy más felhasználók is csatlakozzanak. A csatlakozott tagok bármikor elhagyhatják a háztartást, a tulajdonos pedig szükség esetén el is távolíthatja őket.



11. ábra: Háztartások lista, valamint háztartás részletes nézete.

4.3 Bevásárlólista

Az alsó navigációs sávon elérhető első képernyő a bevásárlólistát jeleníti meg. Az elemek két csoportba rendezve láthatók: az első a megvásárolni kívánt, a második pedig a már megvásárolt tételeket tartalmazza. Egy listaelem megérintésével az adott tétel átkerül a *Megvenni* csoportból a *Megvéve* csoportba. A már megvásárolt elemek a bevásárlókosár ikon segítségével helyezhetők vissza a *Megvenni* csoportba. Elemeket csak a *Megvenni* csoportból lehet törölni, a szemetes ikon segítségével.



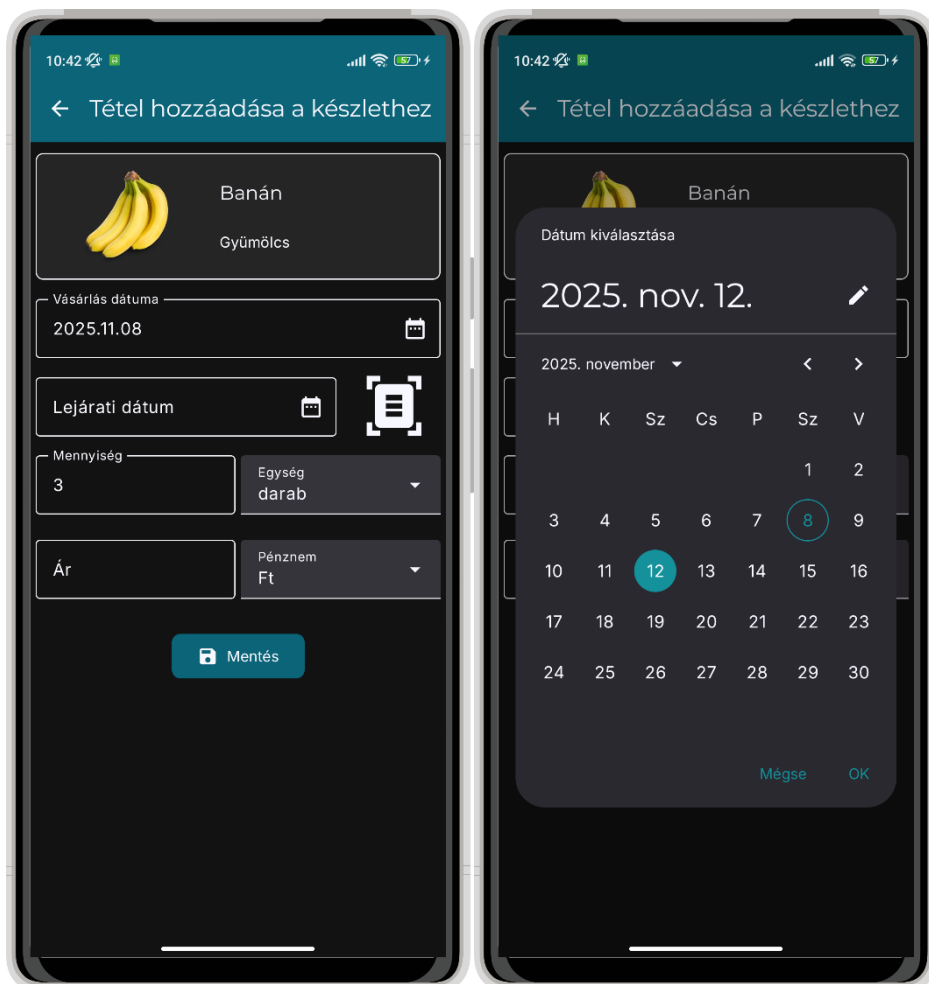
12. ábra: Bevásárlólista nézet, illetve új listaelem felvétele képernyő. Ha egy listaelemhez nem tartozik kép, akkor helyette a kategóriájának ikonja jelenik meg.

A képernyő alján található sávban látható a bevásárlólista *Megvenni* csoportjában lévő elemek összesített költsége, becsült kiadás néven. Ennek előnye, hogy a felhasználók közelítő becslést kaphatnak a vásárlásaik során keletkező kiadásaikról. A termékárak más felhasználók által megadott adatokból származnak. Egy adott termék árának kiszámításakor a megadott darabszámú legfrissebb bejegyzést veszi figyelembe a rendszer, ezekből számítódik a medián, így biztosítva a naprakész költségbecslést.

Új listaelemet a jobb alsó sarokban található plusz gombbal lehet hozzáadni, amely kétféle opciót kínál. A villám ikonnal jelzett gomb a gyors elemfelvételt teszi lehetővé: ilyenkor egy dialógusablakban megadható a termék neve, valamint opcionálisan a mennyiség és a mértékegység. Ez a funkció különösen hasznos, ha a felhasználó még nem ismeri pontosan a megvásárolni kívánt termék konkrét adatait.

A másik gomb egy új képernyőre navigál, ahol a felhasználó kiválaszthatja a terméket a keresősáv segítségével, a *szkennelés* ikonnal beolvashatja a termék

vonalkódját, vagy létrehozhat egy új terméket a *Létrehozás* gombbal. A termék kiválasztását követően itt szükséges megadni a kívánt mennyiséget és mértékegységet, majd a *Mentés* gombbal véglegesíthető az új bevásárlólista elem létrehozása.



13. ábra: A hozzáadás képernyő a módosítható mezőkkel, illetve a dátum választó.

A már megvásárolt termékek jobb oldalán található nyíl ikonra koppintva megjelenik a *Tétel hozzáadása a készlethez* képernyő. Amennyiben a bevásárlólista-elemhez korábban nem lett megadva konkrét termék vagy mennyiség, ezek megadása ezen a felületen kötelező. A termék kiválasztásához itt is elérhetők a korábban ismertetett lehetőségek: keresés, vonalkód szkennelése, új termék létrehozása. Ha ezek az adatok már rendelkezésre állnak, a mennyiség szükség esetén utólagosan is módosítható.

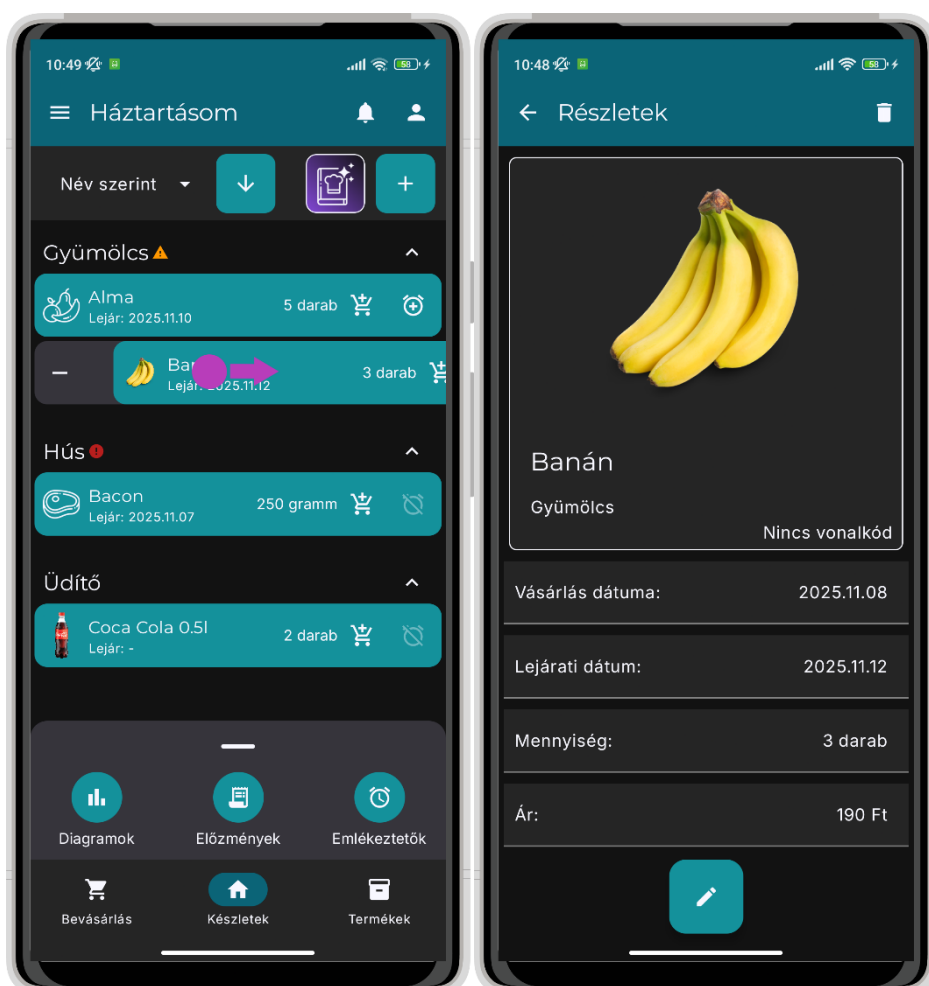
A konkrét terméken és a mennyiségén kívül kötelező megadni a vásárlás dátumát a hozzá tartozó dátumválasztó segítségével, amelyben az aktuális nap alapértelmezetten ki van választva, ezzel is gyorsítva az elemek hozzáadását.

Ezen kívül lehetőség van a termék lejáratí dátumának és árának rögzítésére is. A lejáratí dátumhoz szintén egy dátumválasztó áll rendelkezésre, illetve a dátum szkennelése is támogatott a *szkennelés* gomb segítségével.

A termék árának megadásakor alapértelmezésként fel van tüntetve az alkalmazás által nyilvántartott medián ár, és amennyiben a termék valóban ennyibe került, egyetlen gombnyomással beállítható. Az ár mellett a pénznem is kiválasztható, itt jelenleg csak a forint az egyetlen támogatott opció.

A *Mentés* gomb megnyomását követően létrejön az új készletelem, a hozzá tartozó bevásárlólista-elem pedig automatikusan törlésre kerül.

4.4 Készletek



14. ábra: Készletek lista, szemléltetve a swipe gesztúravezérlelt, illetve egy elem részletes nézete.

Az alsó navigációs sáv második menüpontja a *Készletek* képernyő, ahol a létrehozott készletelemek kategóriák szerint csoportosítva jelennek meg, minden termék

pontosan egy kategóriába tartozik. A kategóriák betűrendben kerülnek megjelenítésre, míg a kategórián belüli termékek név vagy lejárat dátum szerint rendezhetők, mindkét esetben választható növekvő vagy csökkenő sorrend.

A kategóriák neve mellett figyelmeztető ikonok jelenhetnek meg, amelyek a bennük található termékek lejárat idejére utalnak. A sárga háromszög ikon arra figyelmeztet, hogy a kategóriában legalább egy termék három napon belül lejár, míg a piros kör ikon azt jelzi, hogy legalább egy termék már lejárt.

A készletek listában megjelenő elemek újra hozzáadhatók a bevásárlólistához a bevásárlókocsi ikon segítségével. A listaelemek támogatják a *swipe* gesztusvezérlést is: balról-jobbra történő húzás esetén az adott elem eltávolításra kerül a listából, jobbról-balra húzás esetén pedig először hozzáadásra kerül a bevásárlólistához, majd csak ezt követően tűnik el. A *swipe* használatával eltávolított készletelemek csak az aktuális készletlistából törölődnek, a készletelőzmények között továbbra is megtekinthetők maradnak.

Egy listaelemre koppintva megnyílik a készletelem részletes nézete. Innen az alul, középen elhelyezett ceruza ikon megérintésével érhető el a módosító nézet. Mind a részletes, mind a módosító nézeten lehetőség van a készletelem törlésére a jobb felső sarokban található szemetes ikon használatával. A törlés megerősítéséhez egy felugró dialógus ablak jelenik meg, amelyben a felhasználó tájékoztatást kap arról, hogy az ilyen módon törölt elemek nemcsak az aktuális listából, hanem a készletelőzményekből is véglegesen eltávolításra kerülnek.

Új készletelem létrehozására a *Készletek* képernyő jobb felső sarkában található plusz gomb szolgál. A megjelenő képernyő felépítése megegyezik a bevásárlólista-elem rögzítésére szolgáló felülettel, azzal a különbséggel, hogy itt lehetőség van lejárat dátum megadására is. A termék ebben az esetben is kiválasztható a vonalkód beolvasásával, valamint a lejárat dátum is rögzíthető manuálisan vagy szkenneléssel.

A képernyő alján található modális alsó panelen további, a készletekhez kapcsolódó képernyők érhetők el. A *Diagramok* képernyő az aktuális elemek alapján vizualizálja a készleteket, az *Előzmények* képernyőn a mindenkor készletelemek tekinthetők meg, az *Emlékeztetők* képernyőn pedig az aktuális elemekhez beállított emlékeztetők érhetők el.

4.4.1 Diagramok

A diagramokat tartalmazó képernyőn jelenleg kétféle diagramot lehet megtekinteni, melyek célja a készletek aktuális állapotának vizuális összefoglalása.



15. ábra: A készletelemek alapján megjelenő diagramok.

A kördiagram kategóriánként csoportosítva jeleníti meg a készletelemeket: minden körcikk egy-egy kategóriát reprezentál, és az adott kategóriába tartozó elemek számát tükrözi. A kategóriák eltérő színekkel vannak megkülönböztetve a könnyebb áttekinthetőség érdekében.

A halmozott oszlopdiagram a készletelemeket először lejárat dátum szerinti intervallumokba, majd ezen belül kategóriák szerint csoportosítja. Az egyes oszlopok egy-egy lejárat időintervallumot jelölnek, melyek az alábbi csoportokat jelentik: lejárat dátum nincs megadva, már lejárt, 0–3, 4–7, 8–14, 15–30 napon belül lejáró, valamint 30 napon túl lejáró termékek.

A diagramokon egy szekciót hosszabb ideig nyomva tartva megjelenik a szekcióhoz tartozó címke, illetve a diagram alatt látható, hogy melyik csoportba pontosan hány elem tartozik.

4.4.2 Készletelőzmények

Az *Előzmények* képernyőn a felhasználó megtekintheti az összes korábbi készletelemet, amelyet az alkalmazás használata során valaha létrehozott. A felső sávban található opciók segítségével szűrhetők a listában megjelenő elemek: az elmúlt 3, 7, illetve 30 nap alapértelmezett szűrési lehetőségei mellett egyéni hónap is kiválasztható. A képernyő alján egy külön sávban megjelenítésre kerül az aktuálisan listázott elemek összköltsége is.



16. ábra: Előzmények képernyő egy kiválasztott hónaphoz tartozó elemekkel, illetve a kategóriánkénti statisztika ugyanehhez a hónaphoz

A képernyő jobb felső sarkában két gomb található, ezek közül a bal oldali, fájl ikonnal ellátott gomb segítségével az aktuálisan megjelenített elemek táblázatként

exportálhatók. Ennek előnye, hogy az adatok az alkalmazáson kívül is elérhetők, így további feldolgozásra vagy elemzésre is lehetőséget biztosítanak.

4.4.2.1 Készletstatisztikák

Az exportálás gomb mellett található, grafikon ikonnal jelölt gomb segítségével érhető el a készletstatisztikák. Ezen a képernyőn a felhasználók különböző összesítő diagramokat tekinthetnek meg a készleteikkel kapcsolatban. Jelenleg két diagram érhető el.

Az első egy oszlopdiagram, amely havi bontásban ábrázolja a kiválasztott évben történt összkiadásokat. A diagram egyes oszlopait hosszan lenyomva látható az adott hónap neve és a hozzá tartozó kiadás összege. A diagram célja, hogy segítse vizualizálni a felhasználók számára, hogy hónapról-hónapra hogyan változtak a kiadásaik.

A második egy kördiagram, amely hasonlóan az aktuális készletekhez tartozó kördiagramhoz, az elemeket kategóriánként csoportosítva jeleníti meg. A különbség, hogy itt a kategóriánkénti összköltség látható a darabszám helyett. Az oszlopdiagrammal megegyezően ez is egy adott hónap adatait mutatja. A diagram célja, hogy a felhasználók átfogó képet kapjanak arról, mely termékkategóriákra költenek többet vagy kevesebbet, ezáltal segítve, hogy tudatosabban kezeljék a kiadásaikat.

4.4.3 Emlékeztetők

A *Készletek* képernyőn minden listaelem mellett található egy ébresztőóra ikon, amely lehetőséget biztosít egyedi emlékeztető beállítására az adott termék lejárat dátumához kapcsolódóan.

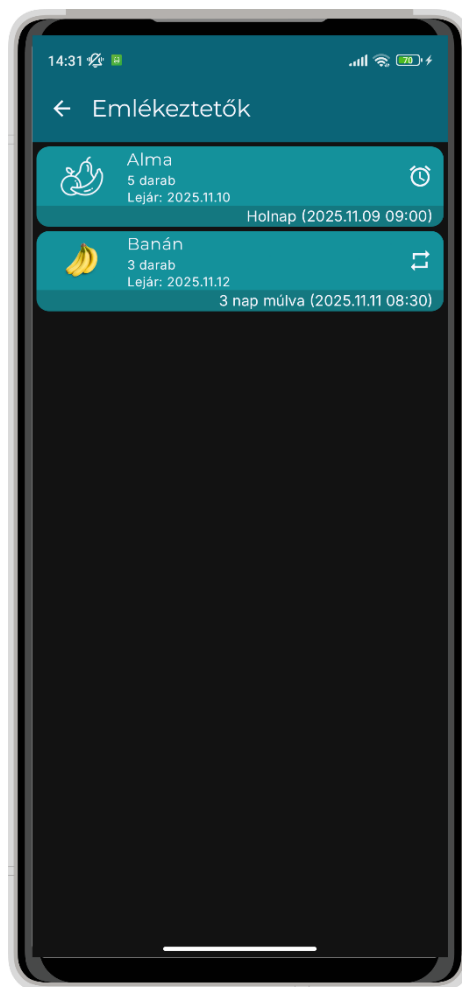
A gomb megnyomásakor egy felugró dialógusablak jelenik meg, ahol beállítható, hogy a felhasználó a termék lejárat előtt hány nappal szeretne értesítést kapni, ez egy és hét nap közötti értékre állítható. Emellett meg kell adni az értesítés kiküldésének időpontját is, amely a napi emlékeztetőkhez hasonlóan 15 perces pontossággal választható ki. Továbbá beállítható, hogy az értesítés egyszeri vagy ismétlődő legyen, ennek alapértelmezett értéke az egyszeri értesítés. Ismétlődő emlékeztető esetén a felhasználó a lejáratig minden nap a megadott időpontban értesítést kap az adott termékről.

A készletek listában, ha egy elemhez emlékeztető be van állítva, akkor a korábbi ébresztőóra ikon helyett egy eltérő ikon jelenik meg, amelyre rákoppintva az emlékeztető kikapcsolható.

4.4.3.1 Emlékeztetők képernyő

Az *Emlékeztetők* képernyőn a készletelemekhez beállított összes emlékeztető egy listában jelenik meg. Minden listaelem tartalmazza a kapcsolódó termék nevét, mennyiségét, illetve lejárat dátumát, az emlékeztető típusát (egyszeri vagy ismétlődő), melyet egy ikon jelez, valamint azt az időpontot, amikor a következő értesítés kiküldése esedékes. A listában megjelenő elemek támogatják a *swipe* gesztusvezérlést is, balról jobbra történő húzással az adott emlékeztető kikapcsolható.

A képernyő célja, hogy a felhasználók központosított helyen kezelhessék a készletelemekhez beállított emlékeztetőiket, megkönnyítve azok áttekintését és menedzselését.

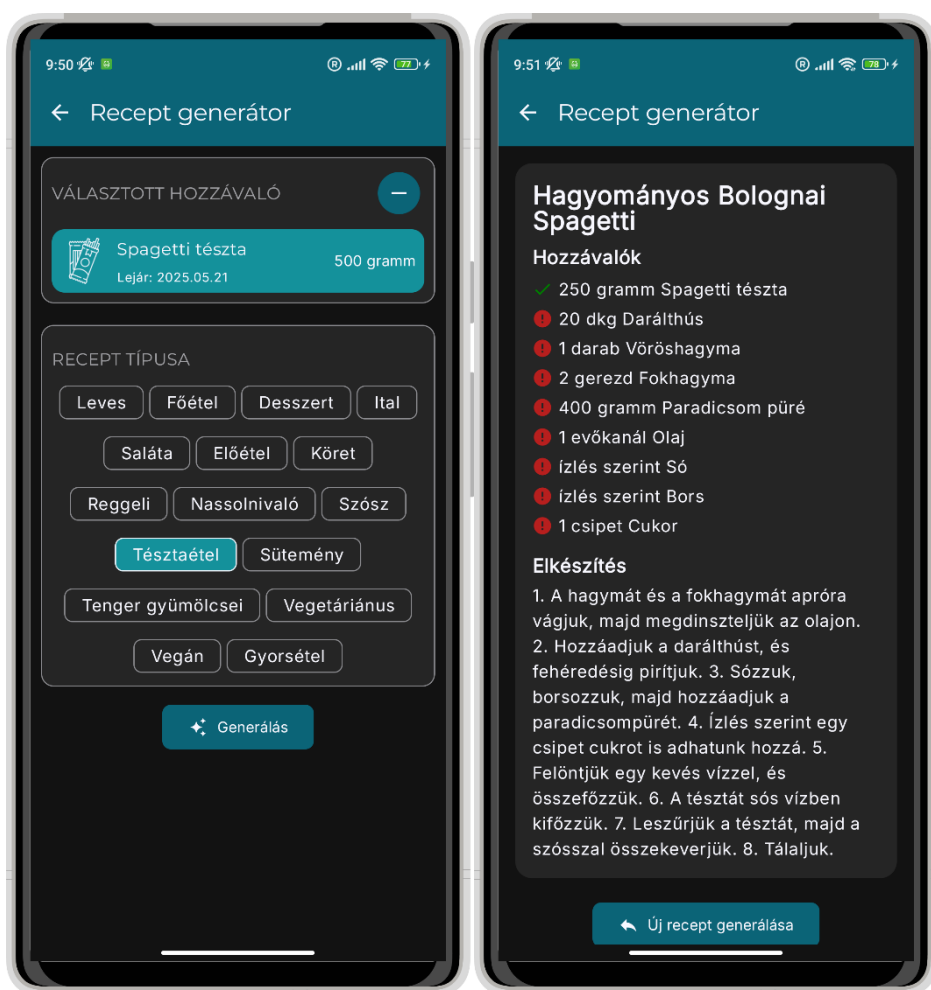


17. ábra: Emlékeztetőket tartalmazó lista

A beállított emlékeztetőket, hasonlóan a napi összefoglalókhoz, push értesítések formájában kapják meg a felhasználók.

4.4.4 Receptgenerálás

A receptgenerálás funkció a *Készletek* képernyőről érhető el az új elem hozzáadása gomb mellett található, szakácskönyv ikonnal ellátott gomb segítségével. A funkció a Gemini AI technológiát alkalmazza a receptek automatikus összeállítására a felhasználó aktuális készlete alapján.



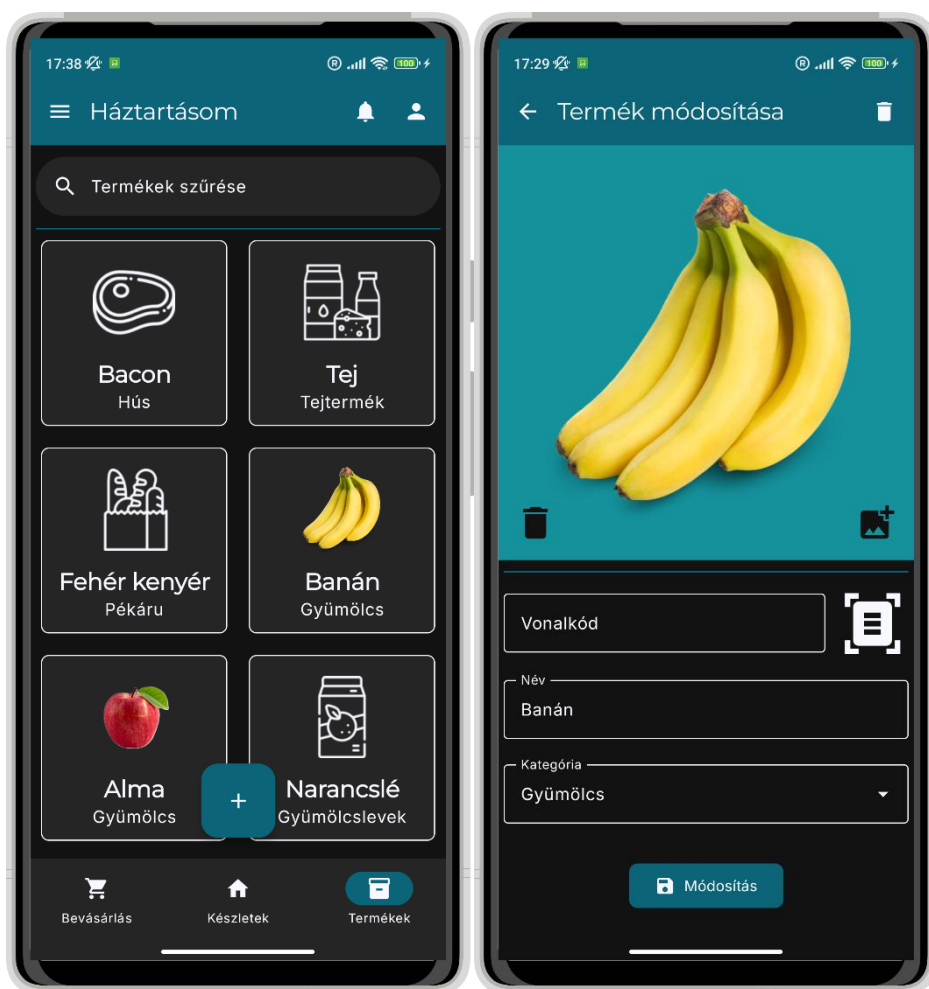
18. ábra: Receptgenerálás konfigurációja, illetve az elkészült recept.

A felhasználók a keresősáv segítségével kiválaszthatnak egy olyan készletelemet, melyet mindenképpen fel szeretnének használni alapanyagként a receptben. Ezt követően megadható a kívánt ételtípus (például főétel, tészaétel vagy sütemény), majd a *Generálás* gomb megnyomásával létrejön a recept.

Az elkészült recept tartalmazza a szükséges hozzávalók listáját, valamint az elkészítési lépéseket. Minden hozzávaló mellett egy ikon jelzi, hogy az adott alapanyag megtalálható-e a felhasználó háztartásában.

4.5 Termékek

Az alsó navigációs sáv utolsó menüpontja a *Termékek* nézet, ahol alapértelmezetten a felhasználó által legutóbb használt termékek jelennek meg. A keresősáv segítségével lehetőség van az összes elérhető termék közötti böngészésre. Amennyiben a felhasználó egy általa létrehozott, de még nem hitelesített termékre kattint, megnyílik a termék szerkesztő nézete. Új termék létrehozására az alul középben elhelyezett plusz gomb segítségével van lehetőség.



19. ábra: Termékek listája, valamint a termék módosító képernyő.

Új termék létrehozásakor kötelező megadni a termék nevét és kiválasztani a megfelelő kategóriát. Ezen kívül opcionálisan megadható a termékhez tartozó vonalkód

a szkennelés támogatásához, valamint lehetőség van képfeltöltésre is. Képet feltölteni a termékképet helyettesítő ikon jobb alsó sarkában található gomb segítségével lehet. A gomb megnyomására megjelenik az Android rendszer beépített vizuális média-választó felülete, amely mivel csak az adott képfájlhoz biztosít hozzáférést, nem igényel külön futásidejű engedélyt. A feltöltött kép a bal alsó sarokban elhelyezett szemetes ikonnal távolítható el.

A termék törlésére a jobb felső sarokban található szemetes ikon használható. Amennyiben az adott termékre létezik hivatkozás, vagyis szerepel bevásárlólista- vagy készletelemként, a törlés előtt egy figyelmeztető párbeszédablak jelenik meg, amely a törlés megerősítését kéri. Ebben az esetben a termékre hivatkozó összes aktív elem is törlésre kerül. Amennyiben a termékhez tartoznak korábban felvett, de már nem aktív készletelemek is, a termék csak logikailag kerül törlésre annak érdekében, hogy az érintett elemek továbbra is megtekinthetők maradjanak a készletelőzményekben.

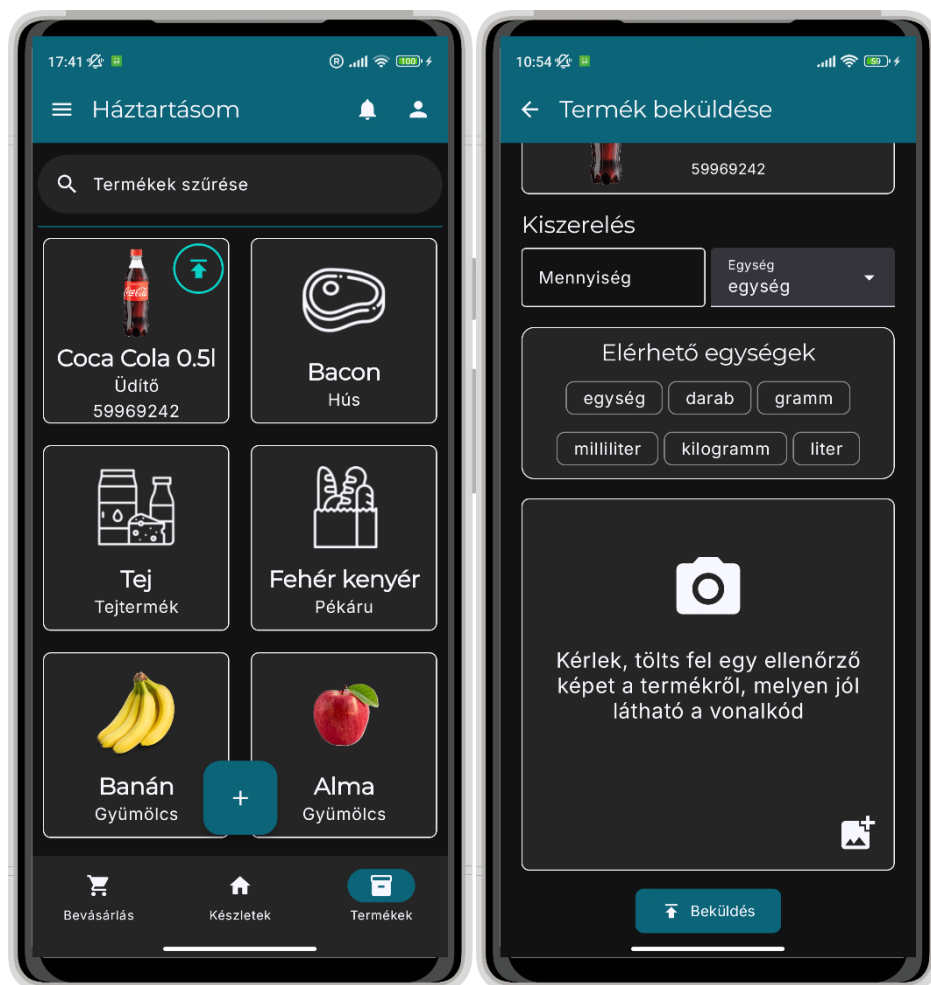
4.5.1 Termék beküldés

Amennyiben egy termék minden tulajdonsága, a vonalkóddal és a termékfotóval együtt kitöltésre került, a *Termékek* lista nézetében megjelenik egy beküldésre szolgáló gomb az adott termékkártya jobb felső sarkában. Ennek megnyomásával a felhasználó a *Termék beküldése* nézetre navigálhat.

A beküldést megelőzően a felhasználónak meg kell adnia a termék kiszерelésének tulajdonságait. Ez azért fontos, mivel a legtöbb előre csomagolt termékből több különböző kiszerelés is elérhető, viszont ezekhez különböző vonalkódok is tartoznak, ezáltal az alkalmazás is külön termékként kezeli őket.

Az elérhető egységek az előre csomagolt termékek esetében szinte mindig a *darab* típust jelenti, viszont a nem előre csomagolt termékeknél, például zöldségek, gyümölcsök, van értelme több opciót is megadni, mint a *kilogramm*, illetve a *gramm*, esetlegesen a *darab*.

A kiszerelés adatai mellett szükséges egy újabb fénykép feltöltése is, amely egyértelműen igazolja, hogy a korábban megadott vonalkód ténylegesen az adott termékhez tartozik. Ennek célja a hitelesítés elősegítése. A szükséges adatok megadása után a termék beküldhető hitelesítésre.



20. ábra: Beküldhető termék a listában, illetve a beküldés képernyő.

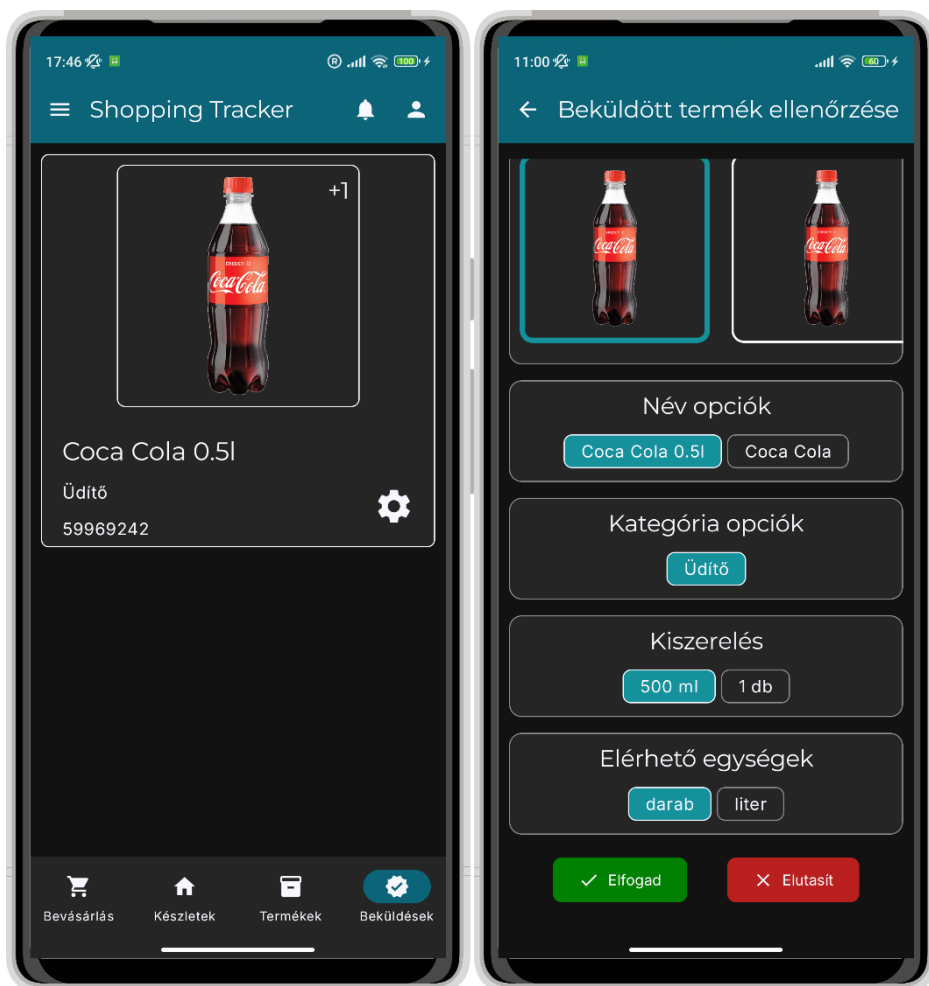
A hitelesített termékek bekerülnek a közös termékadatbázisba, így azok minden felhasználó számára elérhetővé válnak az alkalmazáson belül. Ez a mechanizmus elősegíti az adatbázis folyamatos bővülését, amely közvetlenül javítja a vonalkód-alapú keresések hatékonyságát.

4.5.2 Termék hitelesítés

Az adminisztrátori jogosultsággal rendelkező felhasználók számára elérhető az alkalmazás alsó navigációs sávjában egy negyedik képernyő, *Beküldések* néven, amely a hitelesítésre váró termékeket tartalmazza. Amennyiben egy adott vonalkódhoz több beküldés is érkezik, ezek csoportosítva jelennek meg.

Egy hitelesítendő termék kártyájának jobb alsó sarkában lévő *fogaskerék* ikon megnyomásával megjelenik a hitelesítő felület. Itt az adminisztrátor megtekintheti az adott vonalkódhoz tartozó összes termék-beküldéshez kapcsolódó hitelesítő képet, valamint a termékhez tartozó további jellemzőket. Amennyiben egy adott tulajdonságra

több alternatíva is elérhető, az adminisztrátor kiválaszthatja a legmegfelelőbbet, és a termék elfogadásával véglegesítheti a hitelesítési folyamatot. Amennyiben egyik opció sem elfogadható, akkor el is tudja utasítani a beküldés(ek)e)t. Egy elutasított terméket nem lehet újra beküldeni hitelesítésre.



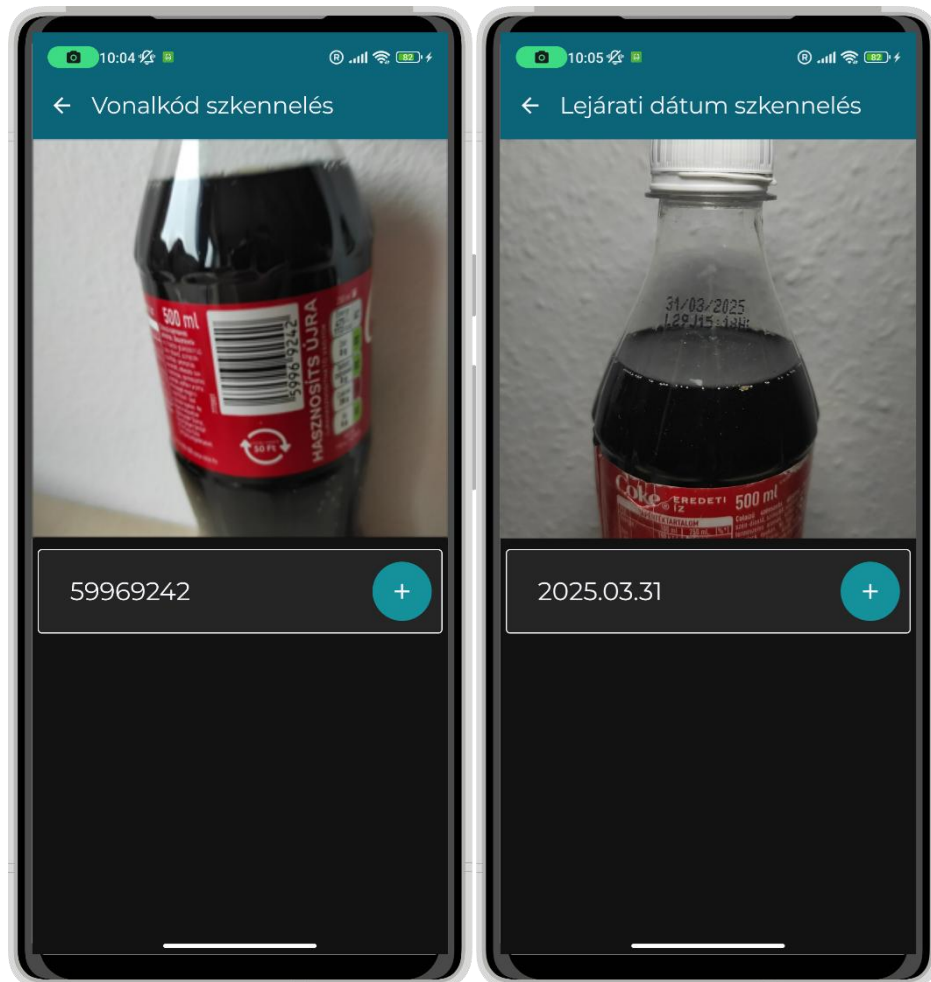
21. ábra: Az adminisztrátorok számára elérhető, hitelesítésre beküldött termékek listája, valamint egy adott vonalkódhoz tartozó beküldések.

A termékek aktuális hitelesítési állapota a hozzájuk tartozó kártyákon különböző ikonok formájában jelenik meg, tájékoztatva a felhasználót, hogy az adott termék beküldhető, beküldött, hitelesített vagy elutasított státuszban van. Az alkalmazás hosszútávú célja az, hogy minél több hitelesített termék legyen, ezzel csökkentve annak szükségességét, hogy a felhasználók saját verziókat hozzanak létre.

4.6 Szkennelő képernyők

A vonalkód- és dátumszkennelő nézetek a különböző képernyőkön elhelyezett szkennelés gombokkal érhetők el. Ezek működéséhez az eszköz kamerájának

használatára van szükség, így a rendszer, amennyiben a felhasználó még nem adott engedélyt, először engedélykérést jelenít meg. Az engedély megadását követően egy kamera-előnézet válik láthatóvá, amely segítségével beolvasható a kívánt vonalkód vagy dátum.



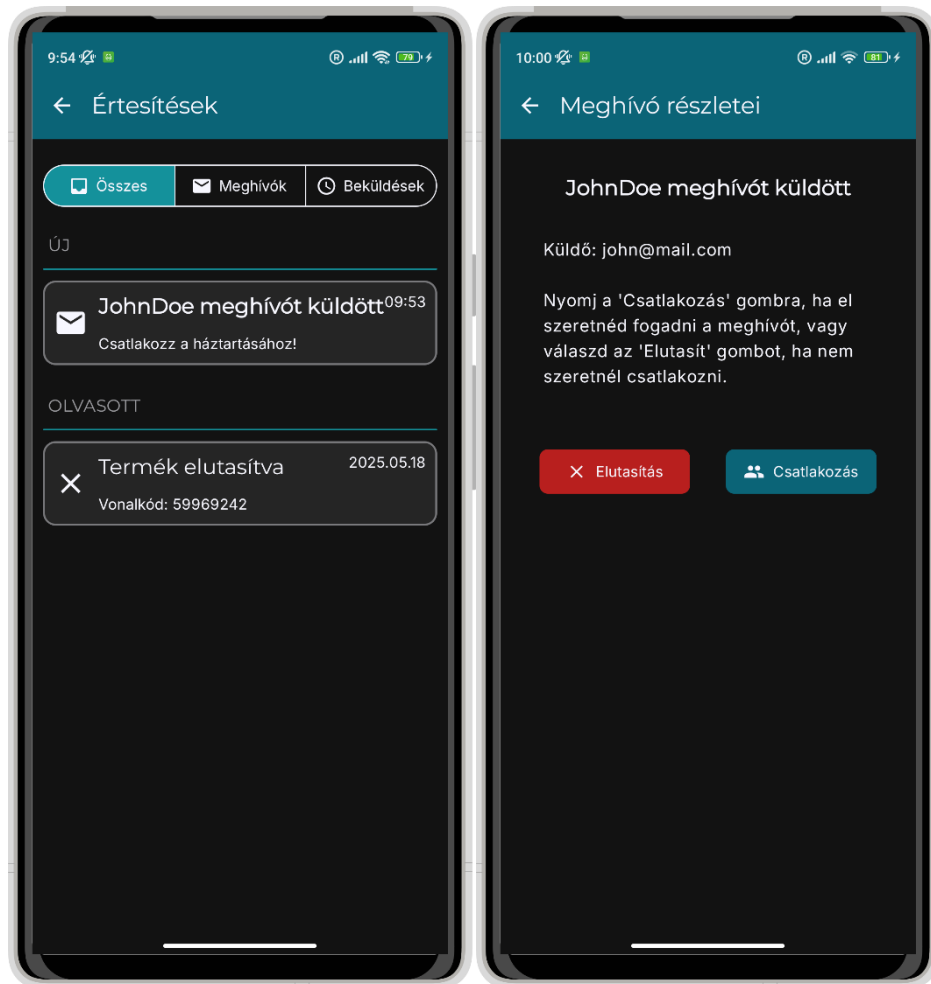
22. ábra: Vonalkód, illetve dátum szkennelő nézet.

A szkennelés során észlelt vonalkódok, illetve dátumok egy listában jelennek meg, ahonnan a felhasználó kiválaszthatja a megfelelő értéket az adott sor mellett található plusz gomb megnyomásával. Erre a képfeldolgozás lehetséges bizonytalansága miatt van szükség, mivel előfordulhat, hogy egyszerre több különböző érték is észlelésre kerül.

4.7 Alkalmazáson belüli értesítések

Az értesítések megtekintésére szolgáló nézetek a főképernyőkről érhetők el, a profilképernyőhöz tartozó gomb mellett elhelyezett harang ikonnal. Az értesítések három

különböző nézetben jeleníthetők meg: az első az összes értesítést tartalmazza, külön kiemelve a még nem megtekintett új elemeket, második kizárólag a háztartási meghívásokat listázza, a harmadik a termékbeküldésekkel kapcsolatos értesítéseket jeleníti meg.



23. ábra: Értesítések képernyő, valamint egy meghívó részletei.

4.7.1 Háztartási meghívók

Amikor egy felhasználó meghív egy másik személyt, hogy csatlakozzon a háztartásához, a rendszer értesítést küld a meghívott fél számára. A meghívó megnyitásakor a felhasználó megtekintheti a meghívást küldő e-mail címét, amely segíti a meghívó hitelesítését és azonosítását. A meghívás elfogadásával a felhasználó automatikusan csatlakozik a megadott háztartáshoz. Amennyiben a felhasználó nem kíván csatlakozni, lehetősége van a meghívás elutasítására is.

4.7.2 Termék beküldések

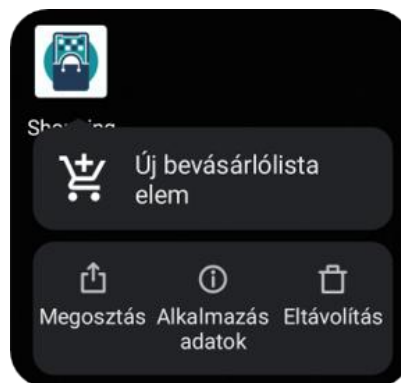
A termékbeküldésekhez kapcsolódó értesítések tájékoztatják a felhasználót az adminisztrátor döntéséről, azaz arról, hogy az általa hitelesítésre beküldött termék elfogadásra vagy elutasításra került. Elfogadás esetén az értesítés megnyitásával megtekinthető a termék hitelesített változata. Ez különösen hasznos, mivel előfordulhat, hogy ugyanarra a termékre több beküldés is érkezett, így a végleges, jóváhagyott termék bizonyos tulajdonságaiban eltérhet a felhasználó által eredetileg beküldött verziótól.

4.8 Kiegészítő funkciók

A következőkben olyan kiegészítő funkciók kerülnek bemutatásra, amelyek bár nem közvetlenül az alkalmazáson belülről érhetők el, mégis hasznos, a felhasználói élményt növelő lehetőségeket kínálnak.

4.8.1 App shortcut

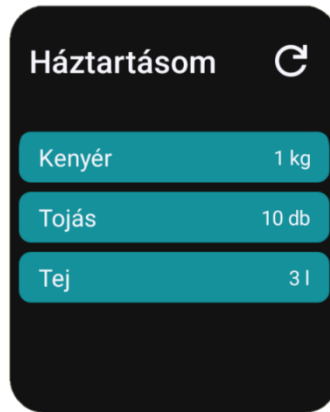
Ez a funkció az alkalmazás ikonjára való hosszan nyomással érhető el. A felhasználó ezzel a funkcióval közvetlenül az *Új bevásárlási tétel* képernyőt nyithatja meg, így gyorsabban vehet fel új elemet a bevásárlólistára.



24. ábra: Gyorselérés funkció

4.8.2 Bevásárlólista widget

Egy olyan kezdőképernyőre helyezhető widgetet biztosít, amelyen a felhasználó az alkalmazás megnyitása nélkül is nyomon követheti az aktuálisan kiválasztott háztartás bevásárlólistájának állapotát. A widget mérete állítható, így könnyebben illeszkedik a képernyőhöz. Alapértelmezetten óránként frissül, az akkumulátor kímélése érdekében, de a beépített frissítés gomb segítségével a tartalom bármikor manuálisan is aktualizálható.



25. ábra: Bevásárlólista widget

4.8.3 Push értesítések

Az alkalmazásban használt push értesítések kezelése teljes mértékben lokálisan történik, a WorkManager komponens segítségével. A napi összefoglaló és az ismétlődő emlékeztetők esetében periodikus háttérfolyamat kerül beütemezésre, míg az egyszeri emlékeztetők egy egyszer lefutó munkafolyamatként kerülnek regisztrálásra. A folyamatok időzítését minden esetben a megadott kezdeti késleltetés határozza meg, amely alapján a rendszer ütemezi azok végrehajtását.

Az implementáció tervezésekor eredetileg a Firebase szolgáltatását szerettem volna használni a push értesítések kezelésére, azonban a szolgáltatás időközben fizetőssé vált. Ennek következtében döntöttem a teljesen lokális megvalósítás mellett, amely nem igényel külső szolgáltatást.

A választott megoldás hátránya, hogy bizonyos gyártók erősebben korlátozzák a nem gyári alkalmazások által létrehozott háttérfolyamatokat, elsősorban az akkumulátor-üzemidő növelésére érdekében. Emiatt előfordulhat, hogy egyes készülékeken a push értesítések nem minden esetben kerülnek kiküldésre, vagy csak jelentős késéssel jutnak el a felhasználóhoz.

4.8.4 Vendégfelhasználók kezelése

Ahogy a felhasználókezelés során már említésre került, az alkalmazás lehetőséget biztosít a regisztráció nélküli, úgynevezett vendégként történő bejelentkezésre is. Ennek viszont különböző korlátai vannak az alkalmazás használata során: bizonyos funkciók nem érhetők el, mint például a receptgenerálás, a

termékbeküldés, illetve a háztartás meghívók küldése és fogadása. Ezek letiltása különböző tervezési megfontolások alapján történtek.

A receptgenerálás mesterséges intelligencia segítségével történik, melynek túlzott használata jelentős költségekkel járhat, illetve jelen esetben a rendelkezésre álló keretek gyors kimerüléséhez vezethet. Éppen ezért célszerűnek találtam ezt a funkciót csak regisztrált felhasználók számára elérhetővé tenni.

A háztartás-meghívók küldése és fogadása szintén olyan funkciók, melyek erősen függenek a felhasználói fióktól. Egy vendégfelhasználóhoz nem tartozik e-mail cím, így nem hívható meg mások által, és ő maga sem képes meghívót küldeni.

A termékbeküldések esetében az adminisztrátorok számára fontos szempont a visszakövethetőség. Ha egy felhasználó rendszeresen hibás vagy akár spam jellegű beküldéseket végez, akkor szükség lehet a fiókjához köthető korlátozások bevezetésére, adott esetben a fiók letiltására, törlésére. Vendégfiók esetén viszont mindez nem lenne megoldható, hiszen ezek a fiókok bármikor törölhetők, és nem kapcsolódik hozzájuk e-mail cím sem, amelyen keresztül azonosíthatók lennének, így a felhasználó gyakorlatilag nyomtalanul eltűnhetne.

A meghívók és a termékbeküldések letiltásának közvetlen következménye, hogy a vendég felhasználók számára az alkalmazáson belüli értesítések nézete sem elérhető, hiszen az itt megtalálható információk kizárólag ezen két forrásból származhatnának, így feleslegessé válik az oldal megjelenítése.

Amennyiben egy vendégfelhasználó mégis megpróbál hozzáférni valamelyik, számára nem elérhető funkcióhoz, az alkalmazás egy figyelmeztető dialógusablakot jelenít meg. Ebben a felhasználó tájékoztatást kap arról, hogy a funkció csak teljes értékű fiókkal használható. A felugró ablak egy gorsgombot is tartalmaz, amely közvetlenül a profilképernyőre irányítja, ahol a vendégfiók átalakítható teljes fiókká. A folyamat sikeres befejezését követően a felhasználó már használhatja is a korábban még nem elérhető funkciókat.

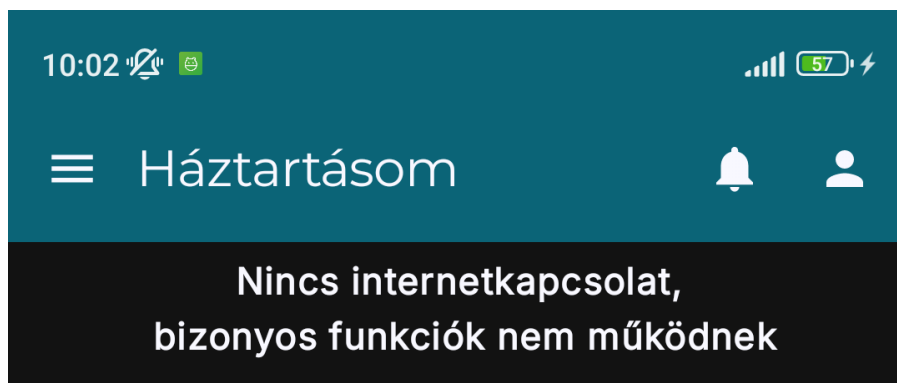


26. ábra: Figyelmeztető dialógus vendégfelhasználók számára.

4.9 Offline működés

Ahogy korábban már említésre került, az alkalmazás internetkapcsolat hiányában is képes bizonyos funkciók ellátására, de a jelenlegi implementációban az offline üzemmód azonban erősen korlátozott. Ez azt jelenti, hogy a felhasználó továbbra is megtekintheti a korábban felvett elemeket, viszont új elemek felvételére, illetve a meglévő elemek módosítására, törlésére nincs lehetőség. Ennek oka, hogy a változtatások mentéséhez szükséges lenne a felhőalapú háttér szolgáltatásokkal való kommunikációra, amely internetkapcsolat nélkül nem biztosítható.

Amennyiben az alkalmazás azt érzékeli, hogy nincs aktív internetkapcsolat, a képernyő felső részén egy figyelmeztető felirat jelenik meg, amely jelzi az offline állapotot. Ennek célja a félreértések elkerülése, mivel a felhasználó így azonnal láthatja, hogy miért nem érhetők el a módosítási és mentési lehetőségek.



27. ábra: Az alkalmazásfejléc alatt, internetkapcsolat hiánya esetén megjelenő üzenet.

5 Tesztelési stratégiák

Az alkalmazás megbízható és helyes működésének egyik elengedhetetlen feltétele a megfelelő tesztelési stratégiák kidolgozása. Ezen projekthez kapcsolódóan három különböző automatizált tesztelési módszert alkalmaztam.

5.1 Platformfüggetlen tesztek

Az első két módszert a platformfüggetlen tesztek kategóriájába sorolhatjuk. Előnyük, hogy futtatásukhoz nem szükséges fizikai eszköz, emulátor vagy szimulátor, így gyorsan végrehajthatók, jól skálázhatók, és egyszerűen beépíthetők a folyamatos integrációs (CI) feldolgozási láncba, ahol a tesztek automatikusan lefutnak a meghatározott események, például a pull request-ek esetén.

5.1.1 Egységtesztek

Az alkalmazásban található osztályok és metódusok helyességének elemi szintű vizsgálata alapvető fontosságú, hiszen ezen komponensek csak akkor tudnak megfelelően együttműködni, ha külön-külön is hibamentesen ellájták a feladatukat. Ennek validálására egységteszteket (unit teszteket) készítettem az alkalmazás egyes komponenseihez. Mivel az adatkezelésben kiemelkedő szerepük van a Repository osztályoknak, ezért elsődlegesen ezek tesztelését valósítottam meg.

Az egyes Repository osztályokhoz tartozó függőségek, melyek lehetnek más Repository-k vagy DataSource osztályok, tesztimplementációkkal vannak helyettesítve, annak érdekében, hogy az adott osztály működését izoláltan lehessen vizsgálni. A tesztimplementációkat ebben az esetben konkrét osztályokkal valósítottam meg, melyek a memóriában tárolt adatokkal imitálják a valós adatforrások működését. A fejlesztés során ezért is volt kiemelten fontos, hogy a különböző implementációk interfészek mögé legyenek rejtve, így csak a megfelelő interfészeket kellett megvalósítani.

5.1.2 Integrációs tesztek

Az adatáramlás megfelelő működését integrációs tesztek segítségével valósítottam meg. Ezek a ViewModel rétegtől kezdődően egészen a DataSource rétegig lefedik az adatfeldolgozási lépéseket, így biztosítva azt, hogy a különböző

adatforrásokból beérkező adatok a megfelelő módon történnek feldolgozásra, és kerülnek eljuttatásra a felhasználói felület rétegébe.

A különböző DataSource osztályok, az egységtesztekhez hasonlóan ebben az esetben is memóriában tárolt adatokat használó tesztimplementációkkal vannak helyettesítve, így a tesztek teljes mértékben függetlenek a valós adatforrásoktól, például a Firebase adatbázistól.

Az integrációs tesztosztályokhoz készítettem egy közös őosztályt is, melynek feladata a különböző függőségek inicializálása az alapértelmezett tesztadatokkal. Természetesen a tesztadatok osztályonként változtathatók, amennyiben azt a tesztesetek megkívánják.

Az aszinkron működés tesztelése érdekében szükség volt a tesztelendő kód módosítására is. Az aszinkron folyamatokat végrehajtó coroutine-ok futtatását a megfelelő szálaikon a végrehajtó (Dispatcher) kezeli, melyet korábban minden osztály önállóan definiált. A tesztelés során azonban szükségessé vált a végrehajtóhoz való hozzáférés, ezért a különböző osztályok immáron a végrehajtót is külső függőségként kapják meg. A valódi működés során ez a Hilt segítségével történik, tesztelés esetén pedig a tesztosztályban definiált tesztvégrehajtó kerül megadásra, lehetővé téve annak felügyeletét is.

5.2 Felhasználói felület tesztek

A felhasználói felületek validálását instrumentált tesztek segítségével hajtottam végre. Az instrumentált tesztek lényege, hogy a kód valós Android-környezetben fut, lehetővé téve a felhasználói interakciók, a felületi komponensek és a rendszerközeli viselkedés pontos ellenőrzését.

A tesztkörnyezetben történő függőséginjektálás megfelelő működése érdekében definiáltam egy egyedi *TestRunner* osztályt, amelyhez egy külön *TestActivity* és *TestApplication* osztály is tartozik. A *TestRunner* feladata, hogy biztosítsa a tesztelt komponensek számára az Android-platformra épülő, kontrollált tesztkörnyezet kialakítását. Ezen keresztül a tesztek izoláltan, a valós alkalmazás működésétől függetlenül futtathatók, miközben a szükséges erőforrások és konfigurációk pontosan elérhetők a tesztosztályok számára.

A felületekhez tartozó tesztosztályok tartalmazzák a tesztadatokat, a tesztimplementációkkal helyettesített függőségeket, a tesztkörnyezet inicializálásáért felelős metódusokat, valamint a konkrét tesztfüggvényeket.

Az instrumentált tesztek hatásköre a megjelenítési rétegre korlátozódik, vagyis elsősorban a felhasználói felületek és a hozzájuk tartozó ViewModel-ek együttműködésének ellenőrzésére szolgálnak. A ViewModel-ek által használt Use Case-ek esetében a valódi megvalósítások helyett tesztimplementációkat alkalmaztam, melyeket ebben az esetben a MockK könyvtár segítségével hoztam létre. Két típust használtam: *mock* és *spy*.

A *mock* (hamis objektum) típusú objektumok teljes mértékben helyettesítik az eredeti függőségeket, így lehetővé teszik azok viselkedésének szimulálását anélkül, hogy a valódi implementáció lefutna. Ennek segítségével ellenőrizhető például, hogy egy adott metódus meghívása megtörtént-e, illetve milyen paraméterekkel történt a hívás. A *mock* esetében csak akkor hajtódik végre bármilyen implementáció, ha azt explicit módon definiáljuk az adott metódushoz. Ezt az objektumtípust elsősorban a Use Case-ek helyettesítésére használtam, mivel így a megjelenítési réteget hatékonyan el tudtam választani az üzleti logikától és az adatrétegtől.

A *spy* (megfigyelő objektum) ezzel szemben a valódi objektumra épül, azonban annak bizonyos viselkedései felüldefiniálhatók. Ez a megközelítés akkor hasznos, ha a teszt során szükség van az eredeti logika részleges megtartására, miközben egyes metódusok működését ellenőrzés vagy módosítás céljából kontrollálni kell. Számomra ez a megoldás különösen a ViewModel-ek esetében bizonyult hasznosnak: alapvetően meg tudtam őrizni a ViewModel valódi működését, ugyanakkor a szükséges pontokon felül tudtam definiálni a viselkedést annak érdekében, hogy a tesztesetek megfelelően lefuthassanak.

5.3 Folyamatos integráció

A tesztek végrehajtását a folyamatos integráció (CI) biztosítja, amelyet a GitHub Actions segítségével valósítottam meg. A CI célja, hogy a forráskódban történő módosítások ellenőrzése és a tesztek futtatása automatizált módon történjen, ezáltal bizonyos hibák már a fejlesztés során felismerhetők legyenek.

Az általam létrehozott CI-folyamat a projekt kódjának fordítását, valamint a platformfüggetlen tesztek futtatását hajtja végre. A fordításhoz szükséges, csak lokálisan tárolt, a verziókezelőbe nem feltöltött titkos azonosítókat és kulcsokat a GitHub Secrets tárolja, ahonnan környezeti változóként jutnak el a futtatókörnyezetbe. A GitHub Actions minden futtatásnál egy egységes, izolált környezetet hoz létre, amely garantálja, hogy az eredmények függetlenek legyenek mindenféle előzetes konfigurációtól.

A CI úgy került konfigurálásra, hogy a folyamat ne minden egyes kódbeli módosítás után induljon el, hanem kizárólag meghatározott eseményeket követően. Ezek az események a fő ágra érkező módosítások, illetve más fejlesztési ágakon végrehajtott változtatások fő ágba történő egyesítése. Ennek köszönhetően elkerülhető a felesleges erőforrás-használat, miközben továbbra is biztosított, hogy minden jelentősebb módosítás megfelelő ellenőrzésen essen át.

6 Összegzés

Ebben a fejezetben szeretném összefoglalni a projekt jelenlegi állapotát, bemutatni a fejlesztés során szerzett tapasztalataimat, valamint kitérni a továbbfejlesztési lehetőségekre is.

6.1 A projekt jelenlegi állapota

A fejlesztés során sikerült megvalósítanom a feladatkiírásban meghatározott követelményekhez tartozó funkciókat, valamint számos olyan kiegészítő megoldást is implementáltam, melyek egyszerűbbé és kényelmesebbé tudják tenni az alkalmazás mindennapos használatát.

A készletek, a bevásárlólista és a termékek kezelése együttesen biztosítják az alkalmazás alapvető működését. Erre épülnek a közös háztartásvezetést támogató funkciók, mint például a háztartások megosztása és az ehhez kapcsolódó meghívók kezelése. A termékadatbázis bővítését és naprakészen tartását a termékbeküldési és hitelesítési folyamatok, valamint az árkövetés teszik lehetővé. A készletek állapotának nyomon követését az emlékeztetők támogatják, melyek segítenek abban, hogy a felhasználók időben értesüljenek a közelgő lejáratokról.

A további kényelmi megoldások, mint a vonalkód- és dátumszkennelés, egyszerűsítik az adatrögzítést. A különböző információs nézetek, diagramok és statisztikák részletes betekintést nyújtanak a készletek aktuális és korábbi állapotába, a kiegészítő funkciók, mint például a widget vagy a hang alapú termékkeresés pedig tovább növelik a felhasználói élményt.

6.2 Projekt értékelése

A projekt elkészítése során szükség volt több, már korábban megismert technológia együttes használatára. Ezek kiegészültek néhány olyannal, melyekkel most ismerkedtem meg, ide tartozik például a Supabase, valamint az ML Kit is.

Véleményem szerint a Supabase kiváló alternatívát nyújt a Firebase helyett, mint felhőalapú adattárolási szolgáltatás, és több szempontból még előnyösebbnek is tartom a használatát. Ingyenes lehetőséget biztosít például a relációs adatbázishoz kapcsolódó triggerek, szerveroldali függvények, valamint saját indexek definiálására, amelyek

nagyban megkönnyítették az adatkezelést és a lekérdezések elkészítését. Emellett az ingyenes fájlátviteli szolgáltatás is nagyon hasznosnak bizonyult, hiszen enélkül nem tudtam volna ilyen jól megvalósítani a képek kezelését az alkalmazásban. Természetesen a Supabase nem váltotta ki teljes mértékben a Firebase használatát, azonban számomra nem is ez volt a cél, a kettő együttes alkalmazásával tudtam elérni a kívánt működést.

Az ML Kit számomra egy érdekes technológia volt. Habár elsőre a használata nem volt feltétlenül egyszerű, a vonalkód- és dátumszkennelési funkciók implementálását nagymértékben megkönnyítette.

Ezen, a korábbiakhoz képest jóval komplexebb projekt fejlesztése során az új technológiák megismerése mellett lehetőségem nyílt mélyebben elmerülni az Android alapú alkalmazásfejlesztésben is. Fejleszthettem programozói képességeimet az Android platformon széles körben elterjedt architektúráis és tervezési minták alkalmazásával, miközben gyakran szükség volt egyedi megoldások használatára is.

A projekt során tapasztalhattam, mennyire fontos a modularizáció és a komponensek közötti függőségek minimalizálása, hogy a rendszer könnyebben tesztelhető, módosítható és bővíthető legyen. Jó példa erre számomra a Jetpack Compose használata a felhasználói felületek készítéséhez, ahol igyekeztem minden felületi komponenst külön elemként kiszervezni, így azok egyszerűen újra felhasználhatók voltak az alkalmazás több pontján is.

Összességében a projekt megvalósítása során rengeteg tudást és tapasztalatot szereztem, melyek alapján a jövőben is szívesen foglalkoznék Android alapú alkalmazásfejlesztéssel.

6.3 Továbbfejlesztési lehetőségek

A projekt elkészítése során több továbbfejlesztési lehetőség is felmerült, részben a saját tapasztalataim, részben pedig annak köszönhetően, hogy több ismerősömnek bemutattam az alkalmazást. Most ezek közül szeretnék néhányat kiemelni.

6.3.1 Papírra írt bevásárlólista digitalizálása

A bevásárlólistát sokan még ma is papírra írják, mivel ez gyors és egyszerű módja a megvásárolni kívánt termékek nyilvántartásának. Ehhez kapcsolódóan felmerült az ötlet, hogy az alkalmazás a vonalkód- és dátumbeolvasás mellett az ML Kit segítségével képes legyen általános, írott szöveg beolvasására a mobil kameráján keresztül. Ezáltal a

papírra írt bevásárlólistán található termékek könnyedén betölthetők lennének az alkalmazásba.

6.3.2 Készletelemek felvétele digitális nyugta alapján

A nagyobb üzletláncok közül sok rendelkezik saját alkalmazással, amelyekben a vásárló fizetésekor a hűségkártya vagy az alkalmazás használatával digitális nyugtát kap. Ez a nyugta a nyomtatott változathoz hasonlóan tartalmazza a megvásárolt termékek listáját. Az ilyen digitális nyugták beolvasása szintén megkönnyítené a felhasználók számára a készletelemek felvételét az alkalmazásba.

6.3.3 Akciós termékek figyelése

A legtöbb vásárló nem csak egyféle boltba jár, és gyakran szeretné követni az akciós termékeket. Ennek megkönnyítésére szolgálna a későbbiekben egy olyan funkció implementálása, amely lekérdezné a digitális újságok tartalmát, és megjelölné az akciós termékeket, feltüntetve az adott üzletlánc nevét és esetleg az akció időtartamát. Így a felhasználók az alkalmazásban egy központosított helyen láthatnák egyszerre több üzletlánc akciós termékeinek listáját.

6.3.4 Általános chat funkció mesterséges intelligenciával

Manapság a mesterséges intelligencia az élet minden területén jelen van, nincs ez másképp a mobilalkalmazások terén sem. Jelenleg az általam fejlesztett alkalmazásban a receptgeneráló funkció használ mesterséges intelligencia szolgáltatást. Ezt az ötletet továbbgondolva egy általános chat funkció is implementálható lenne, amelyen keresztül a felhasználók kérdéseket tehetnének fel a készleteikkel kapcsolatban a mesterséges intelligenciának. Ez a funkció lehetővé tenné, hogy a felhasználók komplexebb információkat is egyszerűen megtudhassanak a készleteikről.

6.3.5 Offline működés kiterjesztése

Érdemes lehet megemlíteni továbbá a teljes körű offline működés támogatását is. Ennek célja az lenne, hogy az alkalmazás internetkapcsolat hiányában ne csak a meglévő adatok megtekintését tegye lehetővé, hanem a létrehozás, módosítás és törlés funkciók is használhatók legyenek. Ehhez a lokális adatbázisnak képesnek kellene lennie a változtatások ideiglenes tárolására, majd internetkapcsolat helyreállítását követően azok szinkronizálására a felhőalapú háttér szolgáltatásokkal.

A megoldás megvalósítása ugyanakkor összetett feladat, mivel olyan szinkronizációs mechanizmust igényel, amely képes a konfliktusok feloldására, biztosítja a műveletek sorrendjét, valamint fenntartja a konzisztens adatállapotot. Mindezt egy olyan környezetben, ahol a több felhasználó által közösen szerkesztett háztartások miatt párhuzamos módosítások is felléphetnek.

Irodalomjegyzék

- [1] Android Developers: *Get started with Jetpack Compose*, <https://developer.android.com/develop/ui/compose/documentation> [Hozzáférés dátuma: 2025. 12. 06.]
- [2] M3 Android: *MDC Android*, <https://m3.material.io/develop/android/mdc-android> [Hozzáférés dátuma: 2025. 11. 29.]
- [3] Google: *Google Fonts*, <https://fonts.google.com/> [Hozzáférés dátuma: 2025. 11. 29.]
- [4] Android Developers: *Navigation with Compose*, <https://developer.android.com/develop/ui/compose/navigation> [Hozzáférés dátuma: 2025. 11. 29.]
- [5] Coil: *Coil Compose*, <https://coil-kt.github.io/coil/compose/> [Hozzáférés dátuma: 2025. 11. 29.]
- [6] Android Developers: *Jetpack Glance*, <https://developer.android.com/develop/ui/compose/glance> [Hozzáférés dátuma: 2025. 11. 29.]
- [7] LottieFiles: *Lottie on Android with Jetpack Compose*, <https://lottiefiles.com/courses/lottie-animations-for-developers/lottie-on-android-with-jetpack-compose> [Hozzáférés dátuma: 2025. 11. 29.]
- [8] Haris Dautović: *Charts*, <https://dautovicharis.github.io/charts/2.0.0/> [Hozzáférés dátuma: 2025. 11. 29.]
- [9] Firebase: *Firebase*, <https://firebase.google.com/> [Hozzáférés dátuma: 2025. 11. 29.]
- [10] Firebase: *Cloud Firestore*, <https://firebase.google.com/docs/firestore> [Hozzáférés dátuma: 2025. 11. 29.]
- [11] MongoDB: *What is NoSQL?*, <https://www.mongodb.com/resources/basics/databases/nosql-explained> [Hozzáférés dátuma: 2025. 11. 29.]
- [12] Supabase: *Supabase*, <https://supabase.com/> [Hozzáférés dátuma: 2025. 11. 29.]
- [13] Supabase: *Database*, <https://supabase.com/docs/guides/database/overview> [Hozzáférés dátuma: 2025. 11. 29.]
- [14] PostgreSQL: *PostgreSQL*, <https://www.postgresql.org/> [Hozzáférés dátuma: 2025. 11. 29.]
- [15] Supabase: *Storage*, <https://supabase.com/docs/guides/storage> [Hozzáférés dátuma: 2025. 11. 29.]

- [16] Firebase: *Firestore Authentication*, <https://firebase.google.com/docs/auth> [Hozzáférés dátuma: 2025. 11. 29.]
- [17] Android Developers: *Credential Manager*, <https://developer.android.com/identity/sign-in/credential-manager> [Hozzáférés dátuma: 2025. 11. 29.]
- [18] Android Developers: *Room*, <https://developer.android.com/jetpack/androidx/releases/room> [Hozzáférés dátuma: 2025. 11. 29.]
- [19] GeeksForGeeks: *What is Object-Relational Mapping (ORM) in DBMS?*, <https://www.geeksforgeeks.org/what-is-object-relational-mapping-orm-in-dbms/> [Hozzáférés dátuma: 2025. 11. 29.]
- [20] SQLite: *What is SQLite?*, <https://www.sqlite.org/> [Hozzáférés dátuma: 2025. 11. 29.]
- [21] Android Developers: *DataStore*, <https://developer.android.com/topic/libraries/architecture/datastore> [Hozzáférés dátuma: 2025. 11. 29.]
- [22] Android Developers: *Camera2 overview*, <https://developer.android.com/media/camera/camera2> [Hozzáférés dátuma: 2025. 11. 29.]
- [23] Google for developers: *ML Kit*, <https://developers.google.com/ml-kit/guides> [Hozzáférés dátuma: 2025. 11. 29.]
- [24] Square Open Source: *Retrofit 2*, <https://square.github.io/retrofit/> [Hozzáférés dátuma: 2025. 11. 29.]
- [25] Android Developers: *Generative AI on Android*, <https://developer.android.com/ai/generativeai> [Hozzáférés dátuma: 2025. 11. 29.]
- [26] Firebase AI Logic: *Gemini API using Firebase AI Logic*, <https://firebase.google.com/docs/ai-logic> [Hozzáférés dátuma: 2025. 11. 29.]
- [27] Google DeepMind: *Gemini 2.5 Flash-Lite*, <https://deepmind.google/technologies/gemini/flash-lite/> [Hozzáférés dátuma: 2025. 11. 29.]
- [28] Apache POI: *The Java API for Microsoft Documents*, <https://poi.apache.org/> [Hozzáférés dátuma: 2025. 11. 29.]
- [29] Android Developers: *Getting started with WorkManager*, <https://developer.android.com/develop/background-work/background-tasks/persistent/getting-started> [Hozzáférés dátuma: 2025. 11. 29.]
- [30] Android Developers: *Dependency injection with Hilt*, <https://developer.android.com/training/dependency-injection/hilt-android> [Hozzáférés dátuma: 2025. 11. 29.]

- [31] Android Developers: *Unit4 rules with AndroidX Test*,
<https://developer.android.com/training/testing/instrumented-tests/androidx-test-libraries/rules> [Hozzáférés dátuma: 2025. 11. 29.]
- [32] MockK: *MockK*, <https://mockk.io/> [Hozzáférés dátuma: 2025. 11. 29.]
- [33] Kotlin: *Kotlin Coroutines Test*,
<https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-test/> [Hozzáférés dátuma: 2025. 11. 29.]
- [34] Android Developers: *Test your Compose layout*,
<https://developer.android.com/develop/ui/compose/testing> [Hozzáférés dátuma: 2025. 11. 29.]
- [35] MockK: *MockK Android*, <https://mockk.io/ANDROID.html> [Hozzáférés dátuma: 2025. 11. 29.]
- [36] Android Developers: *Hilt testing guide*,
<https://developer.android.com/training/dependency-injection/hilt-testing>
[Hozzáférés dátuma: 2025. 11. 29.]
- [37] GitHub Docs: *About GitHub and Git*, <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git> [Hozzáférés dátuma: 2025. 11. 29.]
- [38] GitHub Docs: *Continuous integration*, <https://docs.github.com/en/actions/get-started/continuous-integration> [Hozzáférés dátuma: 2025. 11. 29.]
- [39] GitHub Docs: *Understaning GitHub Actions*,
<https://docs.github.com/en/actions/get-started/understand-github-actions>
[Hozzáférés dátuma: 2025. 11. 29.]
- [40] Android Developers: *Guide to app architecture*,
<https://developer.android.com/topic/architecture> [Hozzáférés dátuma: 2025. 11. 29.]
- [41] CleanCoder: *Clean Architecture*, <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> [Hozzáférés dátuma: 2025. 11. 29.]
- [42] Microsoft Learn: *Model-View-ViewModel (MVVM)*,
<https://learn.microsoft.com/hu-hu/dotnet/architecture/maui/mvvm> [Hozzáférés dátuma: 2025. 11. 29.]
- [43] Medium: *Android MVVM*, <https://medium.com/@dheerubhadoria/android-mvvm-how-to-use-mvvm-in-android-example-7dec84a1fb73> [Hozzáférés dátuma: 2025. 11. 29.]